

Qiskit SDK

v1.3.0 (latest)

API index

Circuit construction

qiskit.circuit

Module overview

AnnotatedOperation

BreakLoopOp

CircuitInstruction

CommutationChecker

ContinueLoopOp

ControlFlowOp

ControlledGate

EquivalenceLibrary

ForLoopOp

Gate

IFElseOp

Instruction

InstructionSet

Operation

Parameter

ParameterExpression

ParameterVector

QuantumCircuit

SwitchCaseOp

WhileLoopOp

qiskit.circuit.classical

qiskit.circuit.classicalfunction

qiskit.circuit.library

qiskit.circuit.singleton

Quantum information (qiskit.quantum_info)

Transpilation

Primitives and providers

Results and visualizations

Serialization (OpenQASM and QPY)

Pulse-level programming

Other

Release notes

QuantumCircuit class

```
class qiskit.circuit.QuantumCircuit(*args, name=None, global_phase=0, metadata=None, input_captures={}, declarations={})
```

GitHub ↗

Bases: `object` ↗

Core Qiskit representation of a quantum circuit.

Note

For more details setting the `QuantumCircuit` in context of all of the data structures that go with it, how it fits into the rest of the `qiskit` package, and the different regimes of quantum-circuit descriptions in Qiskit, see the module-level documentation of `qiskit.circuit`

Circuit attributes

`QuantumCircuit` has a small number of public attributes, which are mostly older functionality. Most of its functionality is accessed through methods.

A small handful of the attributes are intentionally mutable, the rest are data attributes that should be considered immutable.

Mutable attribute	Summary
<code>global_phase</code>	The global phase of the circuit, measured in radians.
<code>metadata</code>	Arbitrary user mapping, which Qiskit will preserve through the transpiler, but otherwise completely ignore.
<code>name</code>	An optional string name for the circuit.

Immutable data attribute	Summary
<code>ancillas</code>	List of <code>AncillaQubit</code> s tracked by the circuit.
<code>calibrations</code>	Custom user-supplied pulse calibrations for individual instructions.
<code>cregs</code>	List of <code>ClassicalRegister</code> s tracked by the circuit.
<code>cbits</code>	List of <code>Cbit</code> s tracked by the circuit.
<code>data</code>	List of individual <code>CircuitInstruction</code> s that make up the circuit.
<code>duration</code>	Total duration of the circuit, added by scheduling transpiler passes.
<code>layout</code>	Hardware layout and routing information added by the transpiler.
<code>num_ancillas</code>	The number of ancilla qubits in the circuit.
<code>num_cbits</code>	The number of cbits in the circuit.
<code>num_captured_vars</code>	Number of captured real-time classical variables.
<code>num_declared_vars</code>	Number of locally declared real-time classical variables in the outer circuit scope.
<code>num_input_vars</code>	Number of input real-time classical variables.
<code>num_parameters</code>	Number of compile-time <code>Parameter</code> s in the circuit.
<code>num_qubits</code>	Number of qubits in the circuit.
<code>num_vars</code>	Total number of real-time classical variables in the outer circuit scope.
<code>op_start_times</code>	Start times of scheduled operations, added by scheduling transpiler passes.
<code>parameters</code>	Ordered set-like view of the compile-time <code>Parameter</code> s tracked by the circuit.
<code>qregs</code>	List of <code>QuantumRegister</code> s tracked by the circuit.
<code>qubits</code>	List of <code>Qubit</code> s tracked by the circuit.
<code>unit</code>	The unit of the <code>duration</code> field.

The core attribute is `data`. This is a sequence-like object that exposes the `CircuitInstruction`s contained in an ordered form. You generally should not mutate this object directly. `QuantumCircuit` is only designed for append-only operations (which should use `append()`). Most operations that mutate circuits in place should be written as transpiler passes (`qiskit.transpiler`).

data

The circuit data (instructions and context).

Returns

a list-like object containing the `CircuitInstruction`s for each instruction.

Return type

QuantumCircuitData

Alongside the `data`, the `global_phase` of a circuit can have some impact on its output, if the circuit is used to describe a `Gate` that may be controlled. This is measured in radians and is directly settable.

global_phase

The global phase of the current circuit scope in radians.

The `name` of a circuit becomes the name of the `Instruction` or `Gate` resulting from `to_instruction()` and `to_gate()` calls, which can be handy for visualizations.

name

Type: `str`

A human-readable name for the circuit.

You can attach arbitrary `metadata` to a circuit. No part of core Qiskit will inspect this or change its behavior based on metadata, but it will be faithfully passed through the transpiler, so you can tag your circuits yourself. When serializing a circuit with QPY (see `qiskit.qpy`), the metadata will be JSON-serialized and you may need to pass a custom serializer to handle non-JSON-compatible objects within it (see `qpy.dump()` for more detail). This field is ignored during export to OpenQASM 2 or 3.

metadata

Arbitrary user-defined metadata for the circuit.

Qiskit will not examine the content of this mapping, but it will pass it through the transpiler and reattach it to the output, so you can track your own metadata.

`QuantumCircuit` exposes data attributes tracking its internal quantum and classical bits and registers. These appear as Python `list`s, but you should treat them as immutable; changing them will *at best* have no effect, and more likely will simply corrupt the internal data of the `QuantumCircuit`.

qregs

Type: `List[qiskit.circuit.quantumregister.QuantumRegister]`

A list of the `QuantumRegister`s in this circuit. You should not mutate this.

cregs

Type: `List[qiskit.circuit.classicalregister.ClassicalRegister]`

A list of the `ClassicalRegister`s in this circuit. You should not mutate this.

qubits

A list of `Qubit`s in the order that they were added. You should not mutate this.

ancillas

A list of `AncillaQubit`s in the order that they were added. You should not mutate this.

cbits

A list of `Cbit`s in the order that they were added. You should not mutate this.

The **compile-time parameters** present in instructions on the circuit are available in `parameters`. This has a canonical order (mostly lexical, except in the case of `ParameterVector`), which matches the order that parameters will be assigned when using the list forms or `assign_parameters()` but also supports `set`-like constant-time membership testing.

parameters

The parameters defined in the circuit.

This attribute returns the `Parameter` objects in the circuit sorted alphabetically. Note that parameters instantiated with a `ParameterVector` are still sorted numerically.

Examples

The snippet below shows that insertion order of parameters does not matter.

```
1 >>> from qiskit.circuit import QuantumCircuit, Parameter
2 >>> a, b, elephant = Parameter("a"), Parameter("b"), Parameter("elephant")
3 >>> circuit = QuantumCircuit(1)
4 >>> circuit.rx(b, 0)
5 >>> circuit.rz(elephant, 0)
6 >>> circuit.ry(a, 0)
7 >>> circuit.parameters # sorted alphabetically!
8 ParameterView([Parameter(a), Parameter(b), Parameter(elephant)])
```

Bear in mind that alphabetical sorting might be unintuitive when it comes to numbers. The literal "10" comes before "2" in strict alphabetical sorting.

```
1 >>> from qiskit.circuit import QuantumCircuit, Parameter
2 >>> angles = [Parameter("angle_1"), Parameter("angle_2"), Parameter("angle_10")]
3 >>> circuit = QuantumCircuit(1)
4 >>> circuit.u(*angles, 0)
5 >>> circuit.draw()
6
7 q: ─── U(angle_1,angle_2,angle_10) ───
8
9 >>> circuit.parameters
10 ParameterView([Parameter(angle_1), Parameter(angle_10), Parameter(angle_2)])
```

To respect numerical sorting, a `ParameterVector` can be used.

On this page

QuantumCircuit class
Circuit attributes
data
global_phase
name
metadata
qregs
qubits
ancillas
cbits
parameters
calibrations
layout
duration
unit
op_start_times
num_ancillas
num_cbits
num_captured_vars
num_declared_vars
num_input_vars
num_parameters
num_qubits
num_vars
Creating new circuits
...init...
copy
copy_empty_like
from_instructions
from_qasm_file
from_qasm_str
Data objects on circuits
Adding data objects
Working with bits and registers
Working with compile-time parameters
Working with real-time typed classical data
Adding operations to circuits
Methods to add general operations
Methods to add standard instructions
Adding control flow to circuits
Converting circuits to single objects
Helper mutation methods
Manual calibration of instructions
Circuit properties
Simple circuit metrics
Accessing scheduling information
Instruction-like methods
control
inverse
power
repeat
reverse_ops
Visualization
draw
decompose
reverse_bits
Internal utilities
cast
cbit_argument_conversion
cis_instances
cis_prefix
qbit_argument_conversion

Was this page helpful?

Yes No

Report a bug or request content on GitHub ↗.

1>>> from qiskit.circuit import QuantumCircuit, Parameter, ParameterVector

2>>> x = ParameterVector("x", 12)

3>>> circuit = QuantumCircuit(4)

4>>> for x_i in x:

5... circuit.cx(x_i, 0)

6>>> circuit.parameters

7ParameterView([

8ParameterVectorElement(x[0]), ParameterVectorElement(x[1]),

9ParameterVectorElement(x[2]), ParameterVectorElement(x[3]),

10..., ParameterVectorElement(x[11])

11])

Returns

The sorted `Parameter` objects in the circuit.

The storage of any manual pulse-level calibrations for individual instructions on the circuit is in `calibrations`

This presents as a `dict`, but should not be mutated directly; use the methods discussed in `Manual calibration of instructions`.

calibrations

Return calibration dictionary.

The custom pulse definition of a given gate is of the form `{gate_name': [(qubits, params), schedule]}`

⛔ Deprecated since version 1.3

The property `qiskit.circuit.quantumcircuit.QuantumCircuit.calibrations` is deprecated as of Qiskit 1.3. It will be removed in Qiskit 2.0. The entire Qiskit Pulse package is being deprecated and will be moved to the Qiskit Dynamics repository: <https://github.com/qiskit-community/qiskit-dynamics>. Note that once removed, `qiskit.circuit.quantumcircuit.QuantumCircuit.calibrations` will have no alternative in Qiskit.

If you have transpiled your circuit, so you have a physical circuit, you can inspect the `layout` attribute for information stored by the transpiler about how the virtual qubits of the source circuit map to the hardware qubits of your physical circuit, both at the start and end of the circuit.

layout

Return any associated layout information about the circuit

This attribute contains an optional `TranspileLayout` object. This is typically set on the output from `transpile()` or `PassManager.run()` to retain information about the permutations caused on the input circuit by transpilation.

There are two types of permutations caused by the `transpile()` function, an initial layout which permutes the qubits based on the selected physical qubits on the `Target` and a final layout which is an output permutation caused by `SwapGate` inserted during routing.

If your circuit was also *scheduled* as part of a transpilation, it will expose the individual timings of each instruction, along with the total `duration` of the circuit.

duration

The total duration of the circuit, set by a scheduling transpiler pass. Its unit is specified by `unit`.

⛔ Deprecated since version 1.3.0

The property `qiskit.circuit.quantumcircuit.QuantumCircuit.duration` is deprecated as of qiskit 1.3.0. It will be removed in Qiskit 2.0.0.

unit

The unit that `duration` is specified in.

⛔ Deprecated since version 1.3.0

The property `qiskit.circuit.quantumcircuit.QuantumCircuit.unit` is deprecated as of qiskit 1.3.0. It will be removed in Qiskit 2.0.0.

op_start_times

Return a list of operation start times.

This attribute is enabled once one of scheduling analysis passes runs on the quantum circuit.

Returns

List of integers representing instruction start times. The index corresponds to the index of instruction in `QuantumCircuit.data`

Raises

AttributeError *⤵* When circuit is not scheduled.

Finally, `QuantumCircuit` exposes several simple properties as dynamic read-only numeric attributes.

num_ancillas

Return the number of ancilla qubits.

num_cbits

Return number of classical bits.

num_captured_vars

The number of real-time classical variables in the circuit marked as captured from an enclosing scope. This is the length of the `iter_captured_vars()` iterable. If this is non-zero, `num_input_vars` must be zero.

num_declared_vars

The number of real-time classical variables in the circuit that are declared by this circuit scope, excluding inputs or captures. This is the length of the `iter_declared_vars()` iterable.

num_input_vars

The number of real-time classical variables in the circuit marked as circuit inputs. This is the length of the `iter_input_vars()` iterable. If this is non-zero, `num_captured_vars` must be zero.

num_parameters

The number of parameter objects in the circuit.

num_qubits

Return number of qubits.

num_vars

The number of real-time classical variables in the circuit. This is the length of the `iter_vars()` iterable.

Creating new circuits

Method	Summary
<code>__init__()</code>	Default constructor of no-instruction circuits.
<code>copy()</code>	Make a complete copy of an existing circuit.
<code>copy_empty_like()</code>	Copy data objects from one circuit into a new one without any instructions.
<code>from_instructions()</code>	Infer data objects needed from a list of instructions.
<code>from_qasm_file()</code>	Legacy interface to <code>qasm2.load()</code>
<code>from_qasm_str()</code>	Legacy interface to <code>qasm2.loads()</code>

The default constructor `QuantumCircuit(...)` produces a circuit with no initial instructions. The arguments to the default constructor can be used to seed the circuit with quantum and classical data storage, and to provide a name, global phase and arbitrary metadata. All of these fields can be expanded later.

__init__

`__init__(regs, name=None, global_phase=0, metadata=None, inputs=(), captures=(), declarations=())`

[GitHub](#)

Default constructor of `QuantumCircuit`.

Parameters

- `regs` (*Register* | *int* *⤵* | *Sequence* | *Bit*) – The registers to be included in the circuit.
 - If a list of `Register` objects, represents the `QuantumRegister` and/or `ClassicalRegister` objects to include in the circuit.
 - For example:

- `QuantumCircuit(QuantumRegister(4))`
 - `QuantumCircuit(QuantumRegister(4), ClassicalRegister(3))`
 - `QuantumCircuit(QuantumRegister(4, "qr0"), QuantumRegister(2, "qr1"))`
 - If a list of `int`, the amount of qubits and/or classical bits to include in the circuit. It can either be a single int for just the number of quantum bits, or 2 ints for the number of quantum bits and classical bits, respectively.
 - For example:

- `QuantumCircuit(4)` *⤵* A `QuantumCircuit` with 4 qubits
 - `QuantumCircuit(4, 3)` *⤵* A `QuantumCircuit` with 4 qubits and 3 classical bits
 - If a list of python lists containing `Bit` objects, a collection of `Bit`s to be added to the circuit.
- `name` (*str* *⤵* | *None*) – the name of the quantum circuit. If not set, an automatically generated string will be assigned.
- `global_phase` (*ParameterValueType*) – The global phase of the circuit in radians.
- `metadata` (*dict* *⤵* | *None*) – Arbitrary key value metadata to associate with the circuit. This gets stored as free-form data in a dict in the `metadata` attribute. It will not be directly used in the circuit.
- `inputs` (*Iterable* | *ExprVar*) – any variables to declare as `Input` runtime variables for this circuit. These should already be existing `ExprVar`s nodes that you build from somewhere else; if you need to create the inputs as well, use `QuantumCircuit.add_input()`. The variables given in this argument will be passed directly to `add_input()`. A circuit cannot have both `inputs` and `captures`.
- `captures` (*Iterable* | *ExprVar*) – any variables that that this circuit scope should capture from a containing scope. The variables given here will be passed directly to `add_capture()`. A circuit cannot have both `inputs` and `captures`.
- `declarations` (*Mapping* | *ExprVar*, *ExprExpr* | *Iterable* | *Tuple* | *ExprVar*, *ExprExpr*) –

any variables that this circuit should declare and initialize immediately. You can order this input so that later declarations depend on earlier ones (including inputs or captures). If you need to depend on values that will be computed later at runtime, use `add_var()` at an appropriate point in the circuit execution.

This argument is intended for convenient circuit initialization when you already have a set of created variables. The variables used here will be directly passed to `add_var()` which you can use directly if this is the first time you are creating the variable.

Raises

- **CircuitError** – if the circuit name, if given, is not valid.
- **CircuitError** – if both `inputs` and `captures` are given.

If you have an existing circuit, you can produce a copy of it using `copy()`, including all its instructions. This is useful if you want to keep partial circuits while extending another, or to have a version you can mutate in-place while leaving the prior one intact.

copy

```
copy(name=None)
```

[GitHub ↗](#)

Copy the circuit.

Parameters

name (*str* ↗) – name to be given to the copied circuit. If None, then the name stays the same.

Returns

a deepcopy of the current circuit, with the specified name

Return type

QuantumCircuit

Similarly, if you want a circuit that contains all the same data objects (bits, registers, variables, etc) but with none of the instructions, you can use `copy_empty_like()`. This is quite common when you want to build up a new layer of a circuit to then use apply onto the back with `compose()` or to do a full rewrite of a circuit's instructions.

copy_empty_like

```
copy_empty_like(name=None, *, vars_mode="allike")
```

[GitHub ↗](#)

Return a copy of self with the same structure but empty.

That structure includes:

- name, calibrations and other metadata
- global phase
- all the qubits and cbits, including the registers
- the realtime variables defined in the circuit, handled according to the `vars` keyword argument.

Warning

If the circuit contains any local variable declarations (those added by the `declarations` argument to the circuit constructor, or using `add_var()`), they may be uninitialized in the output circuit. You will need to manually add store instructions for them (see `Store` and `QuantumCircuit.store()`) to initialize them.

Parameters

- **name** (*str* ↗ / None) – Name for the copied circuit. If None, then the name stays the same.
- **vars_mode** (*Literal*["like", "captures", "drop"]) – The mode to handle realtime variables in.

allike

The variables in the output circuit will have the same declaration semantics as in the original circuit. For example, `inputs` variables in the source will be `inputs` variables in the output circuit.

captures

All variables will be converted to captured variables. This is useful when you are building a new layer for an existing circuit that you will want to `compose()` onto the base, since `compose()` can inline captures onto the base circuit (but not other variables).

drop

The output circuit will have no variables defined.

Returns

An empty copy of self.

Return type

QuantumCircuit

In some cases, it is most convenient to generate a list of `CircuitInstruction`s separately to an entire circuit context, and then to build a circuit from this. The `from_instructions()` constructor will automatically capture all `Qubit` and `Cbit` instances used in the instructions, and create a new `QuantumCircuit` object that has the correct resources and all the instructions.

from_instructions

```
static from_instructions(instructions, *, qubits=(), cbits=(), name=None, global_phase=0, metadata=None)
```

[GitHub ↗](#)

Construct a circuit from an iterable of `CircuitInstruction`s.

Parameters

- **instructions** (*Iterable*[*CircuitInstruction* | *tuple* ↗ (*qiskit.circuit.Instruction*) | *tuple* ↗ (*qiskit.circuit.Instruction*, *Iterable*[*Qubit*]) | *tuple* ↗ (*qiskit.circuit.Instruction*, *Iterable*[*Qubit*], *Iterable*[*Cbit*])]) – The instructions to add to the circuit.
- **qubits** (*Iterable*[*Qubit*]) – Any qubits to add to the circuit. This argument can be used, for example, to enforce a particular ordering of qubits.
- **cbits** (*Iterable*[*Cbit*]) – Any classical bits to add to the circuit. This argument can be used, for example, to enforce a particular ordering of classical bits.
- **name** (*str* ↗ / None) – The name of the circuit.
- **global_phase** (*ParameterValueType*) – The global phase of the circuit in radians.
- **metadata** (*dict* ↗ / None) – Arbitrary key value metadata to associate with the circuit.

Returns

The quantum circuit.

Return type

QuantumCircuit

`QuantumCircuit` also still has two constructor methods that are legacy wrappers around the importers in `qiskit.qasm2`. These automatically apply the **legacy compatibility settings** of `load()` and `loads()`.

from_qasm_file

```
static from_qasm_file(path)
```

[GitHub ↗](#)

Read an OpenQASM 2.0 program from a file and convert to an instance of `QuantumCircuit`.

Parameters

path (*str* ↗) – Path to the file for an OpenQASM 2 program

Returns

The QuantumCircuit object for the input OpenQASM 2.

Return type

QuantumCircuit

See also

`qasm2.load()` the complete interface to the OpenQASM 2 importer.

from_qasm_str

```
static from_qasm_str(qasm_str)
```

[GitHub ↗](#)

Convert a string containing an OpenQASM 2.0 program to a `QuantumCircuit`.

Parameters

qasm_str (*str* ↗) – A string containing an OpenQASM 2.0 program.

Returns

The QuantumCircuit object for the input OpenQASM 2

Return type

QuantumCircuit

See also

`qasm2.loads()` the complete interface to the OpenQASM 2 importer.

Data objects on circuits

Adding data objects

Method	Adds this kind of data
<code>add_bits()</code>	<code>Qubit</code> s and <code>Cbit</code> s.
<code>add_register()</code>	<code>QuantumRegister</code> and <code>ClassicalRegister</code> .
<code>add_var()</code>	<code>Var</code> nodes with local scope and initializers.
<code>add_input()</code>	<code>Var</code> nodes that are treated as circuit inputs.
<code>add_capture()</code>	<code>Var</code> nodes captured from containing scopes.
<code>add_uninitialized_var()</code>	<code>Var</code> nodes with local scope and undefined state.

Typically you add most of the data objects (`Qubit`, `Cbit`, `ClassicalRegister`, etc) to the circuit as part of using the `__init__()` default constructor, or `copy_empty_like()`. However, it is also possible to add these afterwards. Typed classical data, such as standalone `Var` nodes (see **Real-time classical computation**), can be both constructed and added with separate methods.

New registerless `Qubit` and `Cbit` objects are added using `add_bits()`. These objects must not already be present in the circuit. You can check if a bit exists in the circuit already using `find_bit()`.

add_bits

```
add_bits(bits)
```

[GitHub ↗](#)

Add Bits to the circuit.

Parameters

bits (*Iterable* ↗ *Bit*) –

Return type

None

Registers are added to the circuit with `add_register()`. In this method, it is not an error if some of the bits are already present in the circuit. In this case, the register will be an "alias" over the bits. This is not generally well-supported by hardware backends; it is probably best to stay away from relying on it. The registers a given bit is in are part of the return of `find_bit()`.

add_register

```
add_register(*regs)
```

GitHub ↗

Add registers.

Parameters

regs (*Register* *int* ↗ | *Sequence* *Bit*) –

Return type

None

Real-time, typed classical data is represented on the circuit by `Var` nodes with a well-defined `Type`. It is possible to instantiate these separately to a circuit (see `Var.new()`), but it is often more convenient to use circuit methods that will automatically manage the types and expression initialization for you. The two most common methods are `add_var()` (locally scoped variables) and `add_input()` (inputs to the circuit).

add_var

```
add_var(name_or_var, /, initial)
```

GitHub ↗

Add a classical variable with automatic storage and scope to this circuit.

The variable is considered to have been "declared" at the beginning of the circuit, but it only becomes initialized at the point of the circuit that you call this method, so it can depend on variables defined before it.

Parameters

- name_or_var** (*str* ↗ | *expr* *Var*) – either a string of the variable name, or an existing instance of `Var` to re-use. Variables cannot shadow names that are already in use within the circuit.
- initial** (*Any* ↗) –
 - the value to initialize this variable with. If the first argument was given as a string name, the type of the resulting variable is inferred from the initial expression; to control this more manually, either use `Var.new()` to manually construct a new variable with the desired type, or use `expr.cast()` to cast the initializer to the desired type.
 - This must be either a `Expr` node, or a value that can be lifted to one using `expr.lift`.

Returns

The created variable. If a `Var` instance was given, the exact same object will be returned.

Raises

CircuitError – if the variable cannot be created due to shadowing an existing variable.

Return type

exprVar

Examples

Define a new variable given just a name and an initializer expression:

```
1 from qiskit.circuit import QuantumCircuit
2
3 qc = QuantumCircuit(2)
4 my_var = qc.add_var("my_var", False)
```

Reuse a variable that may have been taken from a related circuit, or otherwise constructed manually, and initialize it to some more complicated expression:

```
1 from qiskit.circuit import QuantumCircuit, QuantumRegister, ClassicalRegister
2 from qiskit.circuit.classical import expr, types
3
4 my_var = expr.Var.new("my_var", types.Uint(8))
5
6 cr1 = ClassicalRegister(8, "c1")
7 cr2 = ClassicalRegister(8, "c2")
8 qc = QuantumCircuit(QuantumRegister(8), cr1, cr2)
9
10 # Get some measurement results into each register.
11 qc.h(8)
12 for i in range(1, 8):
13     qc.cx(0, i)
14 qc.measure(range(8), cr1)
15
16 qc.reset(range(8))
17 qc.h(8)
18 for i in range(1, 8):
19     qc.cx(0, i)
20 qc.measure(range(8), cr2)
21
22 # Now when we add the variable, it is initialized using the real-time state of the
23 # two classical registers we measured into above.
24 qc.add_var(my_var, expr.bit_and(cr1, cr2))
```

add_input

```
add_input(name_or_var: str, type: Type, /) * Var
```

```
add_input(name_or_var: Var, type: None * None, /) * Var
```

GitHub ↗

Register a variable as an input to the circuit.

Parameters

- name_or_var** – either a string name, or an existing `Var` node to use as the input variable.
- type** – if the name is given as a string, then this must be a `Type` to use for the variable. If the variable is given as an existing `Var`, then this must not be given, and will instead be read from the object itself.

Returns

the variable created, or the same variable as was passed in.

Raises

CircuitError – if the variable cannot be created due to shadowing an existing variable.

In addition, there are two lower-level methods that can be useful for programmatic generation of circuits. When working interactively, you will most likely not need these; most uses of `add_uninitialized_var()` are part of `copy_empty_like()` and most uses of `add_capture()` would be better off using the **control-flow builder interface**.

add_uninitialized_var

```
add_uninitialized_var(var, /)
```

GitHub ↗

Add a variable with no initializer.

In most cases, you should use `add_var()` to initialize the variable. To use this function, you must already hold a `Var` instance, as the use of the function typically only makes sense in copying contexts.

Warning

Qiskit makes no assertions about what an uninitialized variable will evaluate to at runtime, and some hardware may reject this as an error.

You should treat this function with caution, and as a low-level primitive that is useful only in special cases of programmatically rebuilding two like circuits.

Parameters

var (*Var*) – the variable to add.

add_capture

```
add_capture(var)
```

GitHub ↗

Add a variable to the circuit that it should capture from a scope it will be contained within.

This method requires a `Var` node to enforce that you've got a handle to one, because you will need to declare the same variable using the same object into the outer circuit.

This is a low-level method, which is only really useful if you are manually constructing control-flow operations. You typically will not need to call this method, assuming you are using the builder interface for control-flow scopes (`with` context-manager statements for `if_test()` and the other scoping constructs). The builder interface will automatically make the inner scopes closures on your behalf by capturing any variables that are used within them.

Parameters

var (*Var*) – the variable to capture from an enclosing scope.

Raises

CircuitError – if the variable cannot be created due to shadowing an existing variable.

Working with bits and registers

A `Bit` instance is, on its own, just a unique handle for circuits to use in their own contexts. If you have got a `Bit` instance and a circuit, just can find the contexts that the bit exists in using `find_bit()` such as its integer index in the circuit and any registers it is contained in.

find_bit

```
find_bit(bit)
```

GitHub ↗

Find locations in the circuit which can be used to reference a given `Bit`.

In particular, this function can find the integer index of a qubit, which corresponds to its hardware index for a transpiled circuit.

Note

The circuit index of a `AncillaQubit` will be its index in `qubits`, not `ancillas`.

Parameters

bit (*Bit*) – The bit to locate.

Returns

A 2-tuple. The first element (`index`) contains the index at which the `Bit` can be found (in either `qubits` or `clbits` depending on its type). The second element (`registers`) is a list of (`register`, `index`) pairs with an entry for each `Register` in the circuit which contains the `Bit` (and the index in the `Register` at which it can be found).

Return type

namedtuple(*int* ↗, List[Tuple(*Register*, *int* ↗)])

Raises

- CircuitError** – If the supplied `Bit` was of an unknown type.
- CircuitError** – If the supplied `Bit` could not be found on the circuit.

Examples

Loop through a circuit, getting the qubit and clbit indices of each operation:


```
1 from qiskit.circuit import QuantumCircuit, Qubit
2
3 qc = QuantumCircuit(3, 3)
4 qc.h(0)
5 qc.cx(0, 1)
6 qc.cx(1, 2)
7 qc.measure([0, 1, 2], [0, 1, 2])
8
9 # The '.qubits' and '.clbits' fields are not integers.
10 assert isinstance(qc.data[0].qubits[0], Qubit)
11 # ... but we can use 'find_bit' to retrieve them.
12 assert qc.find_bit(qc.data[0].qubits[0]).index == 0
13
14 single = [
15     (
16         instruction.operation.name,
17         [qc.find_bit(bit).index for bit in instruction.qubits],
18         [qc.find_bit(bit).index for bit in instruction.clbits],
19     )
20     for instruction in qc.data
21 ]
```

Similarly, you can query a circuit to see if a register has already been added to it by using `has_register()`

has_register

`has_register(register)`

[GitHub ↗](#)

Test if this circuit has the register `r`.

Parameters

register (*Register*) – a quantum or classical register.

Returns

True if the register is contained in this circuit.

Return type

`bool` ↗

Working with compile-time parameters

[See also](#)

Compile-time parametrization

A more complete discussion of what compile-time parametrization is, and how it fits into Qiskit's data model.

Unlike bits, registers, and real-time typed classical data, compile-time symbolic parameters are not manually added to a circuit. Their presence is inferred by being contained in operations added to circuits and the global phase. An ordered list of all parameters currently in a circuit is at [QuantumCircuit.parameters](#)

The most common operation on `Parameter` instances is to replace them in symbolic operations with some numeric value, or another symbolic expression. This is done with `assign_parameters()`

assign_parameters

```
assign_parameters(parameters: Mapping[Parameter, ParameterExpression | float] |
Iterable[ParameterExpression | float], inplace: Literal[False] = False, *, flat_input:
bool = False, strict: bool = True) → QuantumCircuit

assign_parameters(parameters: Mapping[Parameter, ParameterExpression | float] |
Iterable[ParameterExpression | float], inplace: Literal[True] = False, *, flat_input:
bool = False, strict: bool = True) → None
```

[GitHub ↗](#)

Assign parameters to new parameters or values.

If `parameters` is passed as a dictionary, the keys should be `Parameter` instances in the current circuit. The values of the dictionary can either be numeric values or new parameter objects.

If `parameters` is passed as a list or array, the elements are assigned to the current parameters in the order of `parameters` which is sorted alphabetically (while respecting the ordering in `ParameterVector` objects).

The values can be assigned to the current circuit object or to a copy of it.

[Note](#)

When `parameters` is given as a mapping, it is permissible to have keys that are strings of the parameter names; these will be looked up using `get_parameter()`. You can also have keys that are `ParameterVector` instances, and in this case, the dictionary value should be a sequence of values of the same length as the vector.

If you use either of these cases, you must leave the setting `flat_input=False` changing this to `True` enables the fast path, where all keys must be `Parameter` instances.

Parameters

- parameters** – Either a dictionary or iterable specifying the new parameter values.
- inplace** – If False, a copy of the circuit with the bound parameters is returned. If True the circuit instance itself is modified.
- flat_input** – If `True` and `parameters` is a mapping type, it is assumed to be exactly a mapping of `[parameter: value]`. By default `False`, the mapping may also contain `ParameterVector` keys that point to a corresponding sequence of values, and these will be unrolled during the mapping, or string keys, which will be converted to `Parameter` instances using `get_parameter()`.
- strict** – If `False`, any parameters given in the mapping that are not used in the circuit will be ignored. If `True` (the default), an error will be raised indicating a logic error.

Raises

- CircuitError** – If parameters is a dict and contains parameters not present in the circuit.
- ValueError** ↗ – If parameters is a list/array and the length mismatches the number of free parameters in the circuit.

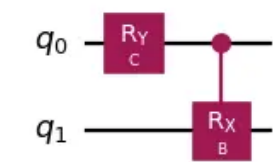
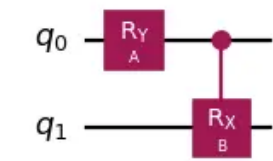
Returns

A copy of the circuit with bound parameters if `inplace` is False, otherwise None.

Examples

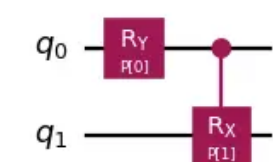
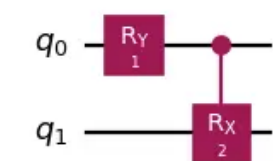
Create a parameterized circuit and assign the parameters in-place.

```
1 from qiskit.circuit import QuantumCircuit, Parameter
2
3 circuit = QuantumCircuit(2)
4 params = [Parameter('A'), Parameter('B'), Parameter('C')]
5 circuit.ry(params[0], 0)
6 circuit.crx(params[1], 0, 1)
7 circuit.draw('mpl')
8 circuit.assign_parameters([params[0]: params[2]], inplace=True)
9 circuit.draw('mpl')
```



Bind the values out-of-place by list and get a copy of the original circuit.

```
1 from qiskit.circuit import QuantumCircuit, ParameterVector
2
3 circuit = QuantumCircuit(2)
4 params = ParameterVector('P', 2)
5 circuit.ry(params[0], 0)
6 circuit.crx(params[1], 0, 1)
7
8 bound_circuit = circuit.assign_parameters([1, 2])
9 bound_circuit.draw('mpl')
10
11 circuit.draw('mpl')
```



The circuit tracks parameters by `Parameter` instances themselves, and forbids having multiple parameters of the same name to avoid some problems when interoperating with OpenQASM or other external formats. You can use `has_parameter()` and `get_parameter()` to query the circuit for a parameter with the given string name.

has_parameter

`has_parameter(name_or_param, /)`

[GitHub ↗](#)

Check whether a parameter object exists in this circuit.

Parameters

name_or_param (*str* ↗ *Parameter*) – the parameter, or name of a parameter to check. If this is a `Parameter` node, the parameter must be exactly the given one for this function to return `True`

Returns

whether a matching parameter is assignable in this circuit.

Return type

`bool` ↗

[See also](#)

`QuantumCircuit.get_parameter()`

Retrieve the `Parameter` instance from this circuit by name.

`QuantumCircuit.has_var()`

A similar method to this, but for run-time `Expr.Var` variables instead of compile-time `Parameter`s.

get_parameter

`get_parameter(name: str, default: T) → Parameter | T`

`get_parameter(name: str, default: Ellipsis) → Parameter`

[GitHub ↗](#)

Retrieve a compile-time parameter that is accessible in this circuit scope by name.

Parameters

- name** – the name of the parameter to retrieve.
- default** – if given, this value will be returned if the parameter is not present. If it is not given, a `KeyError` ↗ is raised instead.

Returns

The corresponding parameter.

Raises

KeyError ↗ – if no default is given, but the parameter does not exist in the circuit.

Examples

Retrieve a parameter by name from a circuit:

```
1 | from qiskit.circuit import QuantumCircuit, Parameter
2 |
3 | my_param = Parameter("my_param")
4 |
5 | # Create a parametrized circuit.
6 | qc = QuantumCircuit(1)
7 | qc.x(my_param, 0)
8 |
9 | # We can use 'my_param' as a parameter, but let's say we've lost the Python object
10 | # and need to retrieve it.
11 | my_param_again = qc.get_parameter("my_param")
12 |
13 | assert my_param is my_param_again
```

Get a variable from a circuit by name, returning some default if it is not present:

```
1 | assert qc.get_parameter("my_param", None) is my_param
2 | assert qc.get_parameter("unknown_param", None) is None
```

See also

`get_var()`

A similar method, but for `Expr.Var` run-time variables instead of `Parameter` compile-time parameters.

Working with real-time typed classical data

See also

`qiskit.circuit.classical`

Module-level documentation for how the variable-, expression- and type-systems work, the objects used to represent them, and the classical operations available.

Real-time classical computation

A discussion of how real-time data fits into the entire `qiskit.circuit` data model as a whole.

Adding data objects

The methods for adding new `Var` variables to a circuit after initialization.

You can retrieve a `Var` instance attached to a circuit by using its variable name using `get_var()` or check if a circuit contains a given variable with `has_var()`.

get_var

`get_var(name: str, default: T) → Var | T`
`get_var(name: str, default: Ellipsis = Ellipsis) → Var`

GitHub ↗

Retrieve a variable that is accessible in this circuit scope by name.

Parameters

- `name` – the name of the variable to retrieve.
- `default` – if given, this value will be returned if the variable is not present. If it is not given, a `KeyError` ↗ is raised instead.

ReturnsThe corresponding variable.

Raises

`KeyError` ↗ – if no default is given, but the variable does not exist.

Examples

Retrieve a variable by name from a circuit:

```
1 | from qiskit.circuit import QuantumCircuit
2 |
3 | # Create a circuit and create a variable in it.
4 | qc = QuantumCircuit()
5 | my_var = qc.add_var("my_var", False)
6 |
7 | # We can use 'my_var' as a variable, but let's say we've lost the Python object and
8 | # need to retrieve it.
9 | my_var_again = qc.get_var("my_var")
10 |
11 | assert my_var is my_var_again
```

Get a variable from a circuit by name, returning some default if it is not present:

```
1 | assert qc.get_var("my_var", None) is my_var
2 | assert qc.get_var("unknown_variable", None) is None
```

See also

`get_parameter()`

A similar method, but for `Parameter` compile-time parameters instead of `Expr.Var` run-time variables.

has_var

`has_var(name: str, /) → bool`

GitHub ↗

Check whether a variable is accessible in this scope.

Parameters

`name_or_var (str ↗ Expr.Var)` – the variable, or name of a variable to check. If this is a `Expr.Var` node, the variable must be exactly the given one for this function to return `True`

Returnswhether a matching variable is accessible.

Return type

`bool` ↗

See also

`QuantumCircuit.get_var()`

Retrieve the `Expr.Var` instance from this circuit by name.

`QuantumCircuit.has_parameter()`

A similar method to this, but for compile-time `Parameter`s instead of run-time `Expr.Var` variables.

There are also several iterator methods that you can use to get the full set of variables tracked by a circuit. At least one of `Iter.input_vars()` and `Iter.captured_vars()` will be empty, as inputs and captures are mutually exclusive. All of the iterators have corresponding dynamic properties on `QuantumCircuit` that contain their length: `num_vars`, `num_input_vars`, `num_captured_vars` and `num_declared_vars`.

iter_vars

`Iter.vars()`

GitHub ↗

Get an iterable over all real-time classical variables in scope within this circuit.

This method will iterate over all variables in scope. For more fine-grained iterators, see `Iter.declared_vars()`, `Iter.input_vars()` and `Iter.captured_vars()`.

Return type

`Iterable` ↗ `[Var]`

iter_input_vars

`Iter.input_vars()`

GitHub ↗

Get an iterable over all real-time classical variables that are declared as inputs to this circuit scope. This excludes locally declared variables (see `Iter.declared_vars()`) and captured variables (see `Iter.captured_vars()`).

Return type

`Iterable` ↗ `[Var]`

iter_captured_vars

`Iter.captured_vars()`

GitHub ↗

Get an iterable over all real-time classical variables that are captured by this circuit scope from a containing scope. This excludes input variables (see `Iter.input_vars()`) and locally declared variables (see `Iter.declared_vars()`).

Return type

`Iterable` ↗ `[Var]`

iter_declared_vars

`Iter.declared_vars()`

GitHub ↗

Get an iterable over all real-time classical variables that are declared with automatic storage duration in this scope. This excludes input variables (see `Iter.input_vars()`) and captured variables (see `Iter.captured_vars()`).

Return type

`Iterable` ↗ `[Var]`

Adding operations to circuits

You can add anything that implements the `Operation` interface to a circuit as a single instruction, though most things you will want to add will be `Instruction` or `Gate` instances.

See also

Operations, instructions and gates

The `qiskit.circuit`-level documentation on the different interfaces that Qiskit uses to define circuit-level instructions.

Methods to add general operations

These are the base methods that handle adding any object, including user-defined ones, onto circuits.

Method	When to use it
<code>append()</code>	Add an instruction as a single object onto a circuit.
<code>_append()</code>	Same as <code>append()</code> , but a low-level interface that elides almost all error checking.
<code>compose()</code>	Inline the instructions from one circuit onto another.
<code>tensor()</code>	Like <code>compose()</code> , but strictly for joining circuits that act on disjoint qubits.

`QuantumCircuit` has two main ways that you will add more operations onto a circuit. Which to use depends on whether you want to add your object as a single "instruction" (`append()`), or whether you want to join the instructions from two circuits together (`compose()`).

A single instruction or operation appears as a single entry in the `[data]` of the circuit, and as a single box when drawn in the circuit visualizers (see `[draw]`). A single instruction is the “unit” that a hardware backend might be defined in terms of (see `[target]`). An `[Instruction]` can come with a `[definition]` which is one rule the transpiler (see `[qiskit.transpiler]`) will be able to fall back on to decompose it for hardware, if needed. An `[Operation]` that is not also an `[Instruction]` can only be decomposed if it has some associated high-level synthesis method registered for it (see `[qiskit.transpiler.passes.synthesis.plugin]`).

A `[QuantumCircuit]` alone is not a single `[Instruction]`; it is rather more complicated, since it can, in general, represent a complete program with typed classical memory inputs and outputs, and control flow. Qiskit’s (and most hardware’s) data model does not yet have the concept of re-usable callable subroutines with virtual quantum operands. You can convert simple circuits that act only on qubits with unitary operations into a `[Gate]` using `[to_gate()]` and simple circuits acting only on qubits and cbits into a `[Instruction]` with `[to_instruction()]`.

When you have an `[Operation]`, `[Instruction]` or `[Gate]` add it to the circuit, specifying the qubit and cbit arguments with `[append()]`.

append

```
append(instruction, qargs=None, cargs=None, *, copy=True)
```

GitHub ↗

Append one or more instructions to the end of the circuit, modifying the circuit in place.

The `[qargs]` and `[cargs]` will be expanded and broadcast according to the rules of the given `[Instruction]` and any non-`[Bit]` specifiers (such as integer indices) will be resolved into the relevant instances.

If a `[CircuitInstruction]` is given, it will be unwrapped, verified in the context of this circuit, and a new object will be appended to the circuit. In this case, you may not pass `[qargs]` or `[cargs]` separately.

Parameters

- instruction** (*Operation / CircuitInstruction*) – `[Instruction]` instance to append, or a `[CircuitInstruction]` with all its context.
- qargs** (*Sequence[QubitSpecifier] / None*) – specifiers of the `[Qubit]`s to attach instruction to.
- cargs** (*Sequence[CbitSpecifier] / None*) – specifiers of the `[Cbit]`s to attach instruction to.
- copy** (*bool* $\rightarrow$ `True`) – If `True` (the default), then the incoming `[Instruction]` is copied before adding it to the circuit if it contains symbolic parameters, so it can be safely mutated without affecting other circuits the same instruction might be in. If you are sure this instruction will not be in other circuits, you can set this `False` for a small speedup.

Returns

a handle to the `[CircuitInstruction]`s that were actually added to the circuit.

Return type

qiskit.circuit.InstructionSet

Raises

CircuitError – if the operation passed is not an instance of `[Instruction]`.

`[append()]` does quite substantial error checking to ensure that you cannot accidentally break the data model of `[QuantumCircuit]`. If you are programmatically generating a circuit from known-good data, you can elide much of this error checking by using the fast-path `[append()]` or `[append()]` but at the risk that the caller is responsible for ensuring they are passing only valid data.

._append

```
._append(instruction: CircuitInstruction, *, _standard_gate: bool) -> CircuitInstruction
```

```
._append(instruction: Operation, qargs: Sequence[Qubit], cargs: Sequence[Cbit]) -> Operation
```

GitHub ↗

Append an instruction to the end of the circuit, modifying the circuit in place.

Warning

This is an internal fast-path function, and it is the responsibility of the caller to ensure that all the arguments are valid; there is no error checking here. In particular:

- all the qubits and cbits must already exist in the circuit and there can be no duplicates in the list.
- any control-flow operations or classically conditioned instructions must act only on variables present in the circuit.
- the circuit must not be within a control-flow builder context.

Note

This function may be used by callers other than `[QuantumCircuit]` when the caller is sure that all error-checking, broadcasting and scoping has already been performed, and the only reference to the circuit the instructions are being appended to is within that same function. In particular, it is not safe to call `[QuantumCircuit._append()]` on a circuit that is received by a function argument. This is because `[QuantumCircuit._append()]` will not recognize the scoping constructs of the control-flow builder interface.

Parameters

- instruction** –
A complete well-formed `[CircuitInstruction]` of the operation and its context to be added.

In the legacy compatibility form, this can be a bare `[Operation]`, in which case `[qargs]` and `[cargs]` must be explicitly given.
- qargs** – Legacy argument for qubits to attach the bare `[Operation]` to. Ignored if the first argument is in the preferential `[CircuitInstruction]` form.
- cargs** – Legacy argument for cbits to attach the bare `[Operation]` to. Ignored if the first argument is in the preferential `[CircuitInstruction]` form.

Returns

a handle to the instruction that was just added.

Return type

CircuitInstruction

In other cases, you may want to join two circuits together, applying the instructions from one circuit onto specified qubits and cbits on another circuit. This “inlining” operation is called `[compose()]` in Qiskit. `[compose()]` is, in general, more powerful than a `[to_instruction()]`-plus-`[append()]` combination for joining two circuits, because it can also link typed classical data together, and allows for circuit control-flow operations to be joined onto another circuit.

The downsides to `[compose()]` are that it is a more complex operation that can involve more rewriting of the operand, and that it necessarily must move data from one circuit object to another. If you are building up a circuit for yourself and raw performance is a core goal, consider passing around your base circuit and having different parts of your algorithm write directly to the base circuit, rather than building a temporary layer circuit.

compose

```
compose(other, qubits=None, cbits=None, front=False, inplace=False, wrap=False, *, copy=True, var_remap=None, inline_captures=False)
```

GitHub ↗

Apply the instructions from one circuit onto specified qubits and/or cbits on another.

Note

By default, this creates a new circuit object, leaving `[self]` untouched. For most uses of this function, it is far more efficient to set `[inplace=True]` and modify the base circuit in-place.

When dealing with realtime variables (`[Expr.Var]` instances), there are two principal strategies for using `[compose()]`.

- The `[other]` circuit is treated as entirely additive, including its variables. The variables in `[other]` must be entirely distinct from those in `[self]` (use `[var_remap]` to help with this), and all variables in `[other]` will be declared anew in the output with matching input/capture/local scoping to how they are in `[other]`. This is generally what you want if you’re joining two unrelated circuits.
- The `[other]` circuit was created as an exact extension to `[self]` to be inlined onto it, including acting on the existing variables in their states at the end of `[self]`. In this case, `[other]` should be created with all these variables to be inlined declared as “captures”, and then you can use `[inline_captures=True]` in this method to link them. This is generally what you want if you’re building up a circuit by defining layers on-the-fly, or rebuilding a circuit using layers taken from itself. You might find the `[vars_node="captures"]` argument to `[copy_empty_like()]` useful to create each layer’s base, in this case.

Parameters

- other** (*qiskit.circuit.Instruction or QuantumCircuit*) – (sub)circuit or instruction to compose onto self. If not a `[QuantumCircuit]` this can be anything that `[append]` will accept.
- qubits** (*list* $\rightarrow$ `[QubitInt]` $\rightarrow$ `int`) – qubits of self to compose onto.
- cbits** (*list* $\rightarrow$ `[CbitInt]` $\rightarrow$ `int`) – cbits of self to compose onto.
- front** (*bool* $\rightarrow$ `False`) – If `True`, front composition will be performed. This is not possible within control-flow builder context managers.
- inplace** (*bool* $\rightarrow$ `False`) – If `True`, modify the object. Otherwise, return composed circuit.
- copy** (*bool* $\rightarrow$ `False`) – If `True` (the default), then the input is treated as shared, and any contained instructions will be copied, if they might need to be mutated in the future. You can set this to `False` if the input should be considered owned by the base circuit, in order to avoid unnecessary copies; in this case, it is not valid to use `[other]` afterward, and some instructions may have been mutated in place.
- var_remap** (*Mapping*) –
mapping to use to rewrite `[Expr.Var]` nodes in `[other]` as they are inlined into `[self]`. This can be used to avoid naming conflicts.

Both keys and values can be given as strings or direct `[Expr.Var]` instances. If a key is a string, it matches any `[Var]` with the same name. If a value is a string, whenever a new key matches a it, a new `[Var]` is created with the correct type. If a value is a `[Var]`, its `[Type]` must exactly match that of the variable it is replacing.

- inline_captures** (*bool* $\rightarrow$ `False`) –
If `True` then all “captured” `[Var]` nodes in the `[other]` `[QuantumCircuit]` are assumed to refer to variables already declared in `[self]` (as any input/capture/local type), and the uses in `[other]` will apply to the existing variables. If you want to build up a layer for an existing circuit to use with `[compose()]` you might find the `[vars_node="captures"]` argument to `[copy_empty_like()]` useful. Any remapping in `[vars_remap]` occurs before evaluating this variable inlining.

If this is `False` (the default), then all variables in `[other]` will be required to be distinct from those in `[self]` and new declarations will be made for them.
- wrap** (*bool* $\rightarrow$ `False`) – If `True`, wraps the other circuit into a gate (or instruction, depending on whether it contains only unitary instructions) before composing it onto self. Rather than using this option, it is almost always better to manually control this yourself by using `[to_instruction()]` or `[to_gate()]` and then call `[append()]`.

Returns

the composed circuit (returns `None` if `inplace==True`).

Return type

QuantumCircuit

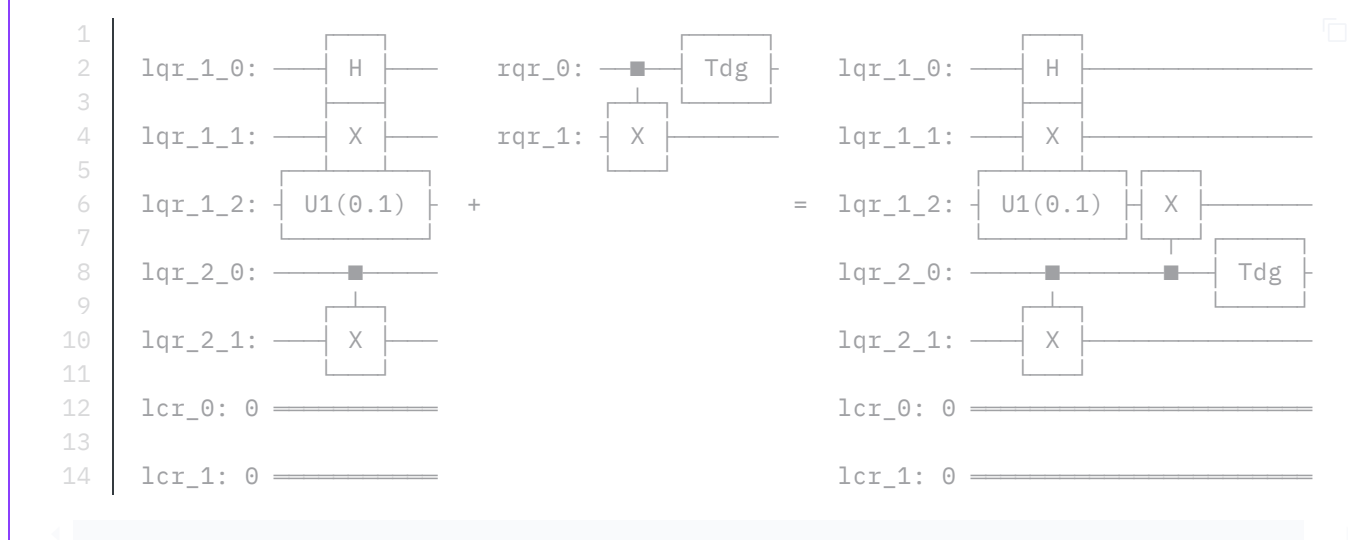
Raises

- CircuitError** – if no correct wire mapping can be made between the two circuits, such as if `[other]` is wider than `[self]`.
- CircuitError** – if trying to emit a new circuit while `[self]` has a partially built control-flow context active, such as the context-manager forms of `[if_test()]`, `[for_loop()]` and `[while_loop()]`.
- CircuitError** – if trying to compose to the front of a circuit when a control-flow builder block is active; there is no clear meaning to this action.

Examples

```
>>> lhs.compose(rhs, qubits=[3, 2], inplace=True)
```





If you are trying to join two circuits that will apply to completely disjoint qubits and cbits, `tensor()` is a convenient wrapper around manually adding bit objects and calling `compose()`.

tensor

`Tensor(other, inplace=False)`

[GitHub ↗](#)

Tensor `self` with `other`

Remember that in the little-endian convention the leftmost operation will be at the bottom of the circuit. See also [the docs](#) for more information.

Parameters

- `other` (*QuantumCircuit*) – The other circuit to tensor this circuit with.
- `inplace` (*bool*) – If `True`, modify the object. Otherwise return composed circuit.

Return type

QuantumCircuit | None

Examples

```
1 from qiskit import QuantumCircuit
2 top = QuantumCircuit(1)
3 top.x(0);
4 bottom = QuantumCircuit(2)
5 bottom.cz(0.2, 0, 1);
6 tensored = bottom.tensor(top)
7 tensored.draw('mpl')
```

Returns

The tensored circuit (returns `None` if `inplace=True`).

Return type

QuantumCircuit

Parameters

- `other` (*QuantumCircuit*) –
- `inplace` (*bool*) –

As some rules of thumb:

- If you have a single `Operation`, `Instruction` or `Gate`, you should definitely use `append()` or `append()`.
- If you have a `QuantumCircuit` that represents a single atomic instruction for a larger circuit that you want to re-use, you probably want to call `to_instruction()` or `to_gate()` and then apply the result of that to the circuit using `append()`.
- If you have a `QuantumCircuit` that represents a larger "layer" of another circuit, or contains typed classical variables or control flow, you should use `compose()` to merge it onto another circuit.
- `tensor()` is wanted far more rarely than either `append()` or `compose()`. Internally, it is mostly a wrapper around `add_bits()` and `compose()`.

Some potential pitfalls to beware of:

- Even if you re-use a custom `Instruction` during circuit construction, the transpiler will generally have to "unroll" each invocation of it to its inner decomposition before beginning work on it. This should not prevent you from using the `to_instruction()` plus `append()` pattern, as the transpiler will improve in this regard over time.
- `compose()` will, by default, produce a new circuit for backwards compatibility. This is more expensive, and not usually what you want, so you should set `inplace=True`.
- Both `append()` and `compose()` (but not `append()`) have a `copy` keyword argument that defaults to `True`. In these cases, the incoming `Operation` instances will be copied if Qiskit detects that the objects have mutability about them (such as taking gate parameters). If you are sure that you will not re-use the objects again in other places, you should set `copy=False` to prevent this copying, which can be a substantial speed-up for large objects.

Methods to add standard instructions

The `QuantumCircuit` class has helper methods to add many of the Qiskit standard-library instructions and gates onto a circuit. These are generally equivalent to manually constructing an instance of the relevant `qiskit.circuit.library` object, then passing that to `append()` with the remaining arguments placed into the `args` and `kwargs` fields as appropriate.

The following methods apply special non-unitary `Instruction` operations to the circuit:

<code>QuantumCircuit</code> method	<code>qiskit.circuit</code> <code>Instruction</code>
<code>barrier()</code>	<code>Barrier</code>
<code>delay()</code>	<code>Delay</code>
<code>initialize()</code>	<code>Initialize</code>
<code>measure()</code>	<code>Measure</code>
<code>reset()</code>	<code>Reset</code>
<code>store()</code>	<code>Store</code>

These methods apply uncontrolled unitary `Gate` instances to the circuit:

<code>QuantumCircuit</code> method	<code>qiskit.circuit.library</code> <code>Gate</code>
<code>dcx()</code>	<code>DCXGate</code>
<code>ecr()</code>	<code>ECRGate</code>
<code>h()</code>	<code>HGate</code>
<code>id()</code>	<code>IGate</code>
<code>iswap()</code>	<code>iSwapGate</code>
<code>ms()</code>	<code>MSGate</code>
<code>p()</code>	<code>PhaseGate</code>
<code>pauli()</code>	<code>PauliGate</code>
<code>prepare_state()</code>	<code>StatePreparation</code>
<code>r()</code>	<code>RGate</code>
<code>rcccx()</code>	<code>RC3XGate</code>
<code>rccx()</code>	<code>RCCXGate</code>
<code>rx()</code>	<code>RXGate</code>
<code>rxn()</code>	<code>RXnGate</code>
<code>ry()</code>	<code>RYGate</code>
<code>ryy()</code>	<code>RYYGate</code>
<code>rz()</code>	<code>RZGate</code>
<code>rzn()</code>	<code>RZnGate</code>
<code>rzz()</code>	<code>RZZGate</code>
<code>s()</code>	<code>SGate</code>
<code>sdg()</code>	<code>SdgGate</code>
<code>swap()</code>	<code>SwapGate</code>
<code>sx()</code>	<code>SXGate</code>
<code>sxdg()</code>	<code>SXdgGate</code>
<code>t()</code>	<code>TGate</code>
<code>tdg()</code>	<code>TdgGate</code>
<code>u()</code>	<code>UGate</code>
<code>unitary()</code>	<code>UnitaryGate</code>
<code>x()</code>	<code>XGate</code>
<code>y()</code>	<code>YGate</code>
<code>z()</code>	<code>ZGate</code>

The following methods apply `Gate` instances that are also controlled gates, so are direct subclasses of `ControlledGate`:

<code>QuantumCircuit</code>	method	<code>qiskit.circuit.library</code>	<code>ControlledGate</code>
<code>ccx()</code>		<code>CCXGate</code>	
<code>ccz()</code>		<code>CCZGate</code>	
<code>ch()</code>		<code>CHGate</code>	
<code>cp()</code>		<code>CPhaseGate</code>	
<code>crx()</code>		<code>CRXGate</code>	
<code>cry()</code>		<code>CRYGate</code>	
<code>crz()</code>		<code>CRZGate</code>	
<code>cs()</code>		<code>CSGate</code>	
<code>csdg()</code>		<code>CSdgGate</code>	
<code>cswap()</code>		<code>CswapGate</code>	
<code>csx()</code>		<code>CSXGate</code>	
<code>cu()</code>		<code>CUGate</code>	
<code>cx()</code>		<code>CXGate</code>	
<code>cy()</code>		<code>CYGate</code>	
<code>cz()</code>		<code>CZGate</code>	

Finally, these methods apply particular generalized multiply controlled gates to the circuit, often with eager syntheses. They are listed in terms of the *base* gate they are controlling, since their exact output is often a synthesized version of a gate.

<code>QuantumCircuit</code>	method	Base	<code>qiskit.circuit.library</code>	<code>Gate</code>
<code>mcp()</code>			<code>PhaseGate</code>	
<code>mcx()</code>			<code>RXGate</code>	
<code>mcry()</code>			<code>RYGate</code>	
<code>mcz()</code>			<code>RZGate</code>	
<code>mcx()</code>			<code>XGate</code>	

The rest of this section is the API listing of all the individual methods; the tables above are summaries whose links will jump you to the correct place.

barrier

`barrier(*qargs, label=None)`

[GitHub ↗](#)

Apply `Barrier` if `qargs` is empty, applies to all qubits in the circuit.

Parameters

- `qargs` (*QubitSpecifier*) – Specification for one or more qubit arguments.
- label (*str ↗* / *None*) – The string label of the barrier.

Returns

handle to the added instructions.

Return type

`qiskit.circuit.InstructionSet`

ccx

`ccx(control_qubit1, control_qubit2, target_qubit, ctrl_state=None)`

[GitHub ↗](#)

Apply `CCXGate`

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- `control_qubit1` (*QubitSpecifier*) – The qubit(s) used as the first control.
- `control_qubit2` (*QubitSpecifier*) – The qubit(s) used as the second control.
- `target_qubit` (*QubitSpecifier*) – The qubit(s) targeted by the gate.
- `ctrl_state` (*str ↗* / *int ↗* / *None*) – The control state in decimal, or as a bitstring (e.g. '1'). Defaults to controlling on the '1' state.

Returns

A handle to the instructions created.

Return type

`InstructionSet`

ccz

`ccz(control_qubit1, control_qubit2, target_qubit, label=None, ctrl_state=None)`

[GitHub ↗](#)

Apply `CCZGate`

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- `control_qubit1` (*QubitSpecifier*) – The qubit(s) used as the first control.
- `control_qubit2` (*QubitSpecifier*) – The qubit(s) used as the second control.
- `target_qubit` (*QubitSpecifier*) – The qubit(s) targeted by the gate.
- label (*str ↗* / *None*) – The string label of the gate in the circuit.
- `ctrl_state` (*str ↗* / *int ↗* / *None*) – The control state in decimal, or as a bitstring (e.g. '10'). Defaults to controlling on the '11' state.

Returns

A handle to the instructions created.

Return type

`InstructionSet`

ch

`ch(control_qubit, target_qubit, label=None, ctrl_states=None)`

[GitHub ↗](#)

Apply `CHGate`

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- `control_qubit` (*QubitSpecifier*) – The qubit(s) used as the control.
- `target_qubit` (*QubitSpecifier*) – The qubit(s) targeted by the gate.
- label (*str ↗* / *None*) – The string label of the gate in the circuit.
- `ctrl_state` (*str ↗* / *int ↗* / *None*) – The control state in decimal, or as a bitstring (e.g. '1'). Defaults to controlling on the '1' state.

Returns

A handle to the instructions created.

Return type

`InstructionSet`

cp

`cp(theta, control_qubit, target_qubit, label=None, ctrl_state=None)`

[GitHub ↗](#)

Apply `CPhaseGate`

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- theta (*ParameterValueType*) – The angle of the rotation.
- `control_qubit` (*QubitSpecifier*) – The qubit(s) used as the control.
- `target_qubit` (*QubitSpecifier*) – The qubit(s) targeted by the gate.
- label (*str ↗* / *None*) – The string label of the gate in the circuit.
- `ctrl_state` (*str ↗* / *int ↗* / *None*) – The control state in decimal, or as a bitstring (e.g. '1'). Defaults to controlling on the '1' state.

Returns

A handle to the instructions created.

Return type

`InstructionSet`

crx

`crx(theta, control_qubit, target_qubit, label=None, ctrl_state=None)`

[GitHub ↗](#)

Apply `CRXGate`

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- theta (*ParameterValueType*) – The angle of the rotation.
- `control_qubit` (*QubitSpecifier*) – The qubit(s) used as the control.
- `target_qubit` (*QubitSpecifier*) – The qubit(s) targeted by the gate.
- label (*str ↗* / *None*) – The string label of the gate in the circuit.
- `ctrl_state` (*str ↗* / *int ↗* / *None*) – The control state in decimal, or as a bitstring (e.g. '1'). Defaults to controlling on the '1' state.

Returns

A handle to the instructions created.

Return type

`InstructionSet`

cry

`cry(theta, control_qubit, target_qubit, label=None, ctrl_state=None)`

[GitHub ↗](#)

Apply `CRYGate`

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- theta (*ParameterValueType*) – The angle of the rotation.

- `control_qubit` (*QubitSpecifier*) – The qubit(s) used as the control.
- `target_qubit` (*QubitSpecifier*) – The qubit(s) targeted by the gate.
- `label` (*str* $\nearrow$ / *None*) – The string label of the gate in the circuit.
- `ctrl_state` (*str* $\nearrow$ /*int* $\nearrow$ / *None*) – The control state in decimal, or as a bitstring (e.g. '1'). Defaults to controlling on the '1' state.

Returns

A handle to the instructions created.

Return type

InstructionSet

crz

```
[crz(theta, control_qubit, target_qubit, label=None, ctrl_state=None)]
```

GitHub ↗

Apply

CRZGate

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- `theta` (*ParameterValue*) – The angle of the rotation.
- `control_qubit` (*QubitSpecifier*) – The qubit(s) used as the control.
- `target_qubit` (*QubitSpecifier*) – The qubit(s) targeted by the gate.
- `label` (*str* $\nearrow$ / *None*) – The string label of the gate in the circuit.
- `ctrl_state` (*str* $\nearrow$ /*int* $\nearrow$ / *None*) – The control state in decimal, or as a bitstring (e.g. '1'). Defaults to controlling on the '1' state.

Returns

A handle to the instructions created.

Return type

InstructionSet

cs

```
[cs(control_qubit, target_qubit, label=None, ctrl_state=None)]
```

GitHub ↗

Apply

CSGate

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- `control_qubit` (*QubitSpecifier*) – The qubit(s) used as the control.
- `target_qubit` (*QubitSpecifier*) – The qubit(s) targeted by the gate.
- `label` (*str* $\nearrow$ / *None*) – The string label of the gate in the circuit.
- `ctrl_state` (*str* $\nearrow$ /*int* $\nearrow$ / *None*) – The control state in decimal, or as a bitstring (e.g. '1'). Defaults to controlling on the '1' state.

Returns

A handle to the instructions created.

Return type

InstructionSet

csdg

```
[csdg(control_qubit, target_qubit, label=None, ctrl_state=None)]
```

GitHub ↗

Apply

CSdgGate

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- `control_qubit` (*QubitSpecifier*) – The qubit(s) used as the control.
- `target_qubit` (*QubitSpecifier*) – The qubit(s) targeted by the gate.
- `label` (*str* $\nearrow$ / *None*) – The string label of the gate in the circuit.
- `ctrl_state` (*str* $\nearrow$ /*int* $\nearrow$ / *None*) – The control state in decimal, or as a bitstring (e.g. '1'). Defaults to controlling on the '1' state.

Returns

A handle to the instructions created.

Return type

InstructionSet

cswap

```
[cswap(control_qubit, target_qubit1, target_qubit2, label=None, ctrl_state=None)]
```

GitHub ↗

Apply

CSwapGate

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- `control_qubit` (*QubitSpecifier*) – The qubit(s) used as the control.
- `target_qubit1` (*QubitSpecifier*) – The qubit(s) targeted by the gate.
- `target_qubit2` (*QubitSpecifier*) – The qubit(s) targeted by the gate.
- `label` (*str* $\nearrow$ / *None*) – The string label of the gate in the circuit.
- `ctrl_state` (*str* $\nearrow$ /*int* $\nearrow$ / *None*) – The control state in decimal, or as a bitstring (e.g. `[1]`). Defaults to controlling on the `[1]` state.

Returns

A handle to the instructions created.

Return type

InstructionSet

csx

```
[csx(control_qubit, target_qubit, label=None, ctrl_state=None)]
```

GitHub ↗

Apply

CSXGate

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- `control_qubit` (*QubitSpecifier*) – The qubit(s) used as the control.
- `target_qubit` (*QubitSpecifier*) – The qubit(s) targeted by the gate.
- `label` (*str* $\nearrow$ / *None*) – The string label of the gate in the circuit.
- `ctrl_state` (*str* $\nearrow$ /*int* $\nearrow$ / *None*) – The control state in decimal, or as a bitstring (e.g. '1'). Defaults to controlling on the '1' state.

Returns

A handle to the instructions created.

Return type

InstructionSet

cu

```
[cu(theta, phi, lam, gamma, control_qubit, target_qubit, label=None, ctrl_state=None)]
```

GitHub ↗

Apply

CUGate

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- `theta` (*ParameterValue*) – The θ rotation angle of the gate.
- `phi` (*ParameterValue*) – The ϕ rotation angle of the gate.
- `lam` (*ParameterValue*) – The λ rotation angle of the gate.
- `gamma` (*ParameterValue*) – The global phase applied of the U gate, if applied.
- `control_qubit` (*QubitSpecifier*) – The qubit(s) used as the control.
- `target_qubit` (*QubitSpecifier*) – The qubit(s) targeted by the gate.
- `label` (*str* $\nearrow$ / *None*) – The string label of the gate in the circuit.
- `ctrl_state` (*str* $\nearrow$ /*int* $\nearrow$ / *None*) – The control state in decimal, or as a bitstring (e.g. '1'). Defaults to controlling on the '1' state.

Returns

A handle to the instructions created.

Return type

InstructionSet

cx

```
[cx(control_qubit, target_qubit, label=None, ctrl_state=None)]
```

GitHub ↗

Apply

CXGate

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- `control_qubit` (*QubitSpecifier*) – The qubit(s) used as the control.
- `target_qubit` (*QubitSpecifier*) – The qubit(s) targeted by the gate.
- `label` (*str* $\nearrow$ / *None*) – The string label of the gate in the circuit.
- `ctrl_state` (*str* $\nearrow$ /*int* $\nearrow$ / *None*) – The control state in decimal, or as a bitstring (e.g. '1'). Defaults to controlling on the '1' state.

Returns

A handle to the instructions created.

Return type

InstructionSet

cy

```
[cy(control_qubit, target_qubit, label=None, ctrl_state=None)]
```

GitHub ↗

Apply

CYGate

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- `control_qubit` (*QubitSpecifier*) – The qubit(s) used as the controls.
- `target_qubit` (*QubitSpecifier*) – The qubit(s) targeted by the gate.
- `label` (*str* $\nearrow$ / *None*) – The string label of the gate in the circuit.
- `ctrl_state` (*str* $\nearrow$ /*int* $\nearrow$ / *None*) – The control state in decimal, or as a bitstring (e.g. '1'). Defaults to controlling on the '1' state.

Returns

A handle to the instructions created.

Return type

InstructionSet

cz

cz(control_qubit, target_qubit, label=None, ctrl_state=None)

GitHub ↗

Apply

CZGate

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- control_qubit (*QubitSpecifier*) – The qubit(s) used as the controls.
- target_qubit (*QubitSpecifier*) – The qubit(s) targeted by the gate.
- label (*str* ↗ / *None*) – The string label of the gate in the circuit.
- ctrl_state (*str* ↗ / *int* ↗ / *None*) – The control state in decimal, or as a bitstring (e.g. '1'). Defaults to controlling on the '1' state.

Returns

A handle to the instructions created.

Return type

InstructionSet

dcx

dcx(qubit1, qubit2)

GitHub ↗

Apply

DCXGate

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- qubit1 (*Qubit* / *QuantumRegister* / *int* ↗ / *slice* ↗ / *Sequence* ↗ (*Qubit* / *int* ↗)) – The qubit(s) to apply the gate to.
- qubit2 (*Qubit* / *QuantumRegister* / *int* ↗ / *slice* ↗ / *Sequence* ↗ (*Qubit* / *int* ↗)) – The qubit(s) to apply the gate to.

Returns

A handle to the instructions created.

Return type

InstructionSet

delay

delay(duration, qarg=None, unit='dt')

GitHub ↗

Apply

Delay

 If qarg is `None` applies to all qubits. When applying to multiple qubits, delays with the same duration will be created.

Parameters

- duration (*int* ↗ *or float* ↗ *or ParameterExpression*) – duration of the delay.
- qarg (*Object*) – qubit argument to apply this delay.
- unit (*str* ↗) – unit of the duration. Supported units: `"s"` `"ms"` `"us"` `"ns"` `"ps"` and `"dt"`. Default is `"dt"` i.e. integer time unit depending on the target backend.

Returns

handle to the added instructions.

Return type

qiskit.circuit.InstructionSet

Raises

CircuitError – if arguments have bad format.

ecr

ecr(qubit1, qubit2)

GitHub ↗

Apply

ECRGate

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- qubit1 (*Qubit* / *QuantumRegister* / *int* ↗ / *slice* ↗ / *Sequence* ↗ (*Qubit* / *int* ↗)) – The qubits to apply the gate to.
- qubit2 (*Qubit* / *QuantumRegister* / *int* ↗ / *slice* ↗ / *Sequence* ↗ (*Qubit* / *int* ↗)) – The qubits to apply the gate to.

Returns

A handle to the instructions created.

Return type

InstructionSet

h

h(qubit)

GitHub ↗

Apply

HGate

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- qubit (*Qubit* / *QuantumRegister* / *int* ↗ / *slice* ↗ / *Sequence* ↗ (*Qubit* / *int* ↗)) – The qubit(s) to apply the gate to.

Returns

A handle to the instructions created.

Return type

InstructionSet

id

id(qubit)

GitHub ↗

Apply

IGate

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- qubit (*Qubit* / *QuantumRegister* / *int* ↗ / *slice* ↗ / *Sequence* ↗ (*Qubit* / *int* ↗)) – The qubit(s) to apply the gate to.

Returns

A handle to the instructions created.

Return type

InstructionSet

initialize

initialize(params, qubits=None, normalize=False)

GitHub ↗

Initialize qubits in a specific state.

Qubit initialization is done by first resetting the qubits to `|0⟩`, followed by calling `StatePreparation` class to prepare the qubits in a specified state. Both these steps are included in the `Initialize` instruction.

Parameters

- params (*Statevector* / *Sequence* / *complex* ↗) / *str* ↗ (*int* ↗) –

The state to initialize to, can be either of the following.
 - Statevector or vector of complex amplitudes to initialize to.
 - Labels of basis states of the Pauli eigenstates Z, X, Y. See `Statevector.from_label()`. Notice the order of the labels is reversed with respect to the qubit index to be applied to. Example label `"01"` initializes the qubit zero to `|1⟩` and the qubit one to `|0⟩`.
 - An integer that is used as a bitmap indicating which qubits to initialize to `|1⟩`. Example: setting params to 5 would initialize qubit 0 and qubit 2 to `|1⟩` and qubit 1 to `|0⟩`.
- qubits (*Sequence* / *QubitSpecifier* / *None*) – Qubits to initialize. If `None` the initialization is applied to all qubits in the circuit.
- normalize (*bool* ↗) – Whether to normalize an input array to a unit vector.

Returns

A handle to the instructions created.

Examples

Prepare a qubit in the state $\frac{(|0\rangle - |1\rangle)}{\sqrt{2}}$.

```
1 | import numpy as np
2 | from qiskit import QuantumCircuit
3 |
4 | circuit = QuantumCircuit(1)
5 | circuit.initialize([1/np.sqrt(2), -1/np.sqrt(2)], 0)
6 | circuit.draw()
```

output:

```
1 |
2 | q_0: ┌───┐
3 |       │ Initialize(0.70711,-0.70711) │
4 |       └───┘
```

Initialize from a string two qubits in the state `|10⟩`. The order of the labels is reversed with respect to qubit index. More information about labels for basis states are in `Statevector.from_label()`.

```
1 | import numpy as np
2 | from qiskit import QuantumCircuit
3 |
4 | circuit = QuantumCircuit(2)
5 | circuit.initialize('01', circuit.qubits)
6 | circuit.draw()
```

output:

```
1 |
2 | q_0: ┌───┐
3 |       │ Initialize(0,1) │
4 | q_1: ──┤─┘
5 |       │
```

Initialize two qubits from an array of complex amplitudes.

```
1 | import numpy as np
2 | from qiskit import QuantumCircuit
3 |
4 | circuit = QuantumCircuit(2)
5 | circuit.initialize([0, 1/np.sqrt(2), -1.j/np.sqrt(2), 0], circuit.qubits)
6 | circuit.draw()
```

output:


```
1 |
2 | q_0: 0
3 |
4 | q_1: 1
5 |
```

Initialize(0,0.70711,-0.70711j,0)

iswap

`iswap(qubit1, qubit2)`

[GitHub ↗](#)

Apply [ISwapGate](#)

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- qubit1 (*Qubit* / *QuantumRegister* / *int* ↗ / *slice* ↗ / *Sequence* ↗ / *Qubit* / *int* ↗) – The qubits to apply the gate to.
- qubit2 (*Qubit* / *QuantumRegister* / *int* ↗ / *slice* ↗ / *Sequence* ↗ / *Qubit* / *int* ↗) – The qubits to apply the gate to.

Returns

A handle to the instructions created.

Return type

InstructionSet

mcp

`mcp(lam, control_qubits, target_qubit, ctrl_state=None)`

[GitHub ↗](#)

Apply [MCPPhaseGate](#)

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- lam (*ParameterValueType*) – The angle of the rotation.
- control_qubits (*Sequence* / *QubitSpecifier*) – The qubits used as the controls.
- target_qubit (*QubitSpecifier*) – The qubit(s) targeted by the gate.
- ctrl_state (*str* ↗ / *int* ↗ / *None*) – The control state in decimal, or as a bitstring (e.g. '1'). Defaults to controlling on the '1' state.

Returns

A handle to the instructions created.

Return type

InstructionSet

mcx

`mcx(theta, q_controls, q_target, use_basis_gates=False)`

[GitHub ↗](#)

Apply Multiple-Controlled X rotation gate

Parameters

- self (*QuantumCircuit*) – The QuantumCircuit object to apply the mcx gate on.
- theta (*float* ↗) – angle theta
- q_controls (*QuantumRegister* or *list* ↗ / *Qubit*) – The list of control qubits
- q_target (*Qubit*) – The target qubit
- use_basis_gates (*bool* ↗) – use p, u, cx

Raises

QiskitError – parameter errors

mcry

`mcry(theta, q_controls, q_target, q_ancilla=None, mode=None, use_basis_gates=False)`

[GitHub ↗](#)

Apply Multiple-Controlled Y rotation gate

Parameters

- self (*QuantumCircuit*) – The QuantumCircuit object to apply the mcry gate on.
- theta (*float* ↗) – angle theta
- q_controls (*list* ↗ / *Qubit*) – The list of control qubits
- q_target (*Qubit*) – The target qubit
- q_ancillae (*QuantumRegister* or *tuple* ↗ / *QuantumRegister* / *int* ↗) – The list of ancillary qubits.
- mode (*string*) – The implementation mode to use
- use_basis_gates (*bool* ↗) – use p, u, cx

Raises

QiskitError – parameter errors

mcz

`mcz(lam, q_controls, q_target, use_basis_gates=False)`

[GitHub ↗](#)

Apply Multiple-Controlled Z rotation gate

Parameters

- self (*QuantumCircuit*) – The QuantumCircuit object to apply the mcz gate on.
- lam (*float* ↗) – angle lambda
- q_controls (*list* ↗ / *Qubit*) – The list of control qubits
- q_target (*Qubit*) – The target qubit
- use_basis_gates (*bool* ↗) – use p, u, cx

Raises

QiskitError – parameter errors

mcx

`mcx(control_qubits, target_qubit, ancilla_qubits=None, mode='noancilla', ctrl_state=None)`

[GitHub ↗](#)

Apply [MCXgate](#)

The multi-cx gate can be implemented using different techniques, which use different numbers of ancilla qubits and have varying circuit depth. These modes are:

- `"noancilla"` Requires 0 ancilla qubits.
- `"recursion"` Requires 1 ancilla qubit if more than 4 controls are used, otherwise 0.
- `"v-chain"` Requires 2 less ancillas than the number of control qubits.
- `"v-chain-dirty"` Same as for the clean ancillas (but the circuit will be longer).

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- control_qubits (*Sequence* / *QubitSpecifier*) – The qubits used as the controls.
- target_qubit (*QubitSpecifier*) – The qubit(s) targeted by the gate.
- ancilla_qubits (*QubitSpecifier* / *Sequence* / *QubitSpecifier* / *None*) – The qubits used as the ancillae, if the mode requires them.
- mode (*str* ↗) – The choice of mode, explained further above.
- ctrl_state (*str* ↗ / *int* ↗ / *None*) – The control state in decimal, or as a bitstring (e.g. '1'). Defaults to controlling on the '1' state.

Returns

A handle to the instructions created.

Raises

- ValueError** ↗ – if the given mode is not known, or if too few ancilla qubits are passed.
- AttributeError** ↗ – if no ancilla qubits are passed, but some are needed.

Return type

InstructionSet

measure

`measure(qubit, cbit)`

[GitHub ↗](#)

Measure a quantum bit (`qubit`) in the Z basis into a classical bit (`cbit`).

When a quantum state is measured, a qubit is projected in the computational (Pauli Z) basis to either $|0\rangle$ or $|1\rangle$. The classical bit `cbit` indicates the result of that projection as a `0` or a `1` respectively. This operation is non-reversible.

Parameters

- qubit (*Qubit* / *QuantumRegister* / *int* ↗ / *slice* ↗ / *Sequence* ↗ / *Qubit* / *int* ↗) – qubit(s) to measure.
- cbit (*Cbit* / *ClassicalRegister* / *int* ↗ / *slice* ↗ / *Sequence* ↗ / *Cbit* / *int* ↗) – classical bit(s) to place the measurement result(s) in.

Returns

handle to the added instructions.

Return type

`qiskit.circuit.InstructionSet`

Raises

CircuitError – if arguments have bad format.

Examples

In this example, a qubit is measured and the result of that measurement is stored in the classical bit (usually expressed in diagrams as a double line):

```
1 | from qiskit import QuantumCircuit
2 | circuit = QuantumCircuit(1, 1)
3 | circuit.h(0)
4 | circuit.measure(0, 0)
5 | circuit.draw()
```

q:

c: 1/0

H

M

It is possible to call `measure` with lists of `qubits` and `cbits` as a shortcut for one-to-one measurement. These two forms produce identical results:

```
1 | circuit = QuantumCircuit(2, 2)
2 | circuit.measure([0,1], [0,1])
```

```
1 | circuit = QuantumCircuit(2, 2)
2 | circuit.measure(0, 0)
3 | circuit.measure(1, 1)
```

Instead of lists, you can use `QuantumRegister` and `ClassicalRegister` under the same logic.


```
1 from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister
2 creg = QuantumRegister(2, "creg")
3 creg = ClassicalRegister(2, "creg")
4 circuit = QuantumCircuit(creg, creg)
5 circuit.measure(creg, creg)

This is equivalent to:

1 circuit = QuantumCircuit(creg, creg)
2 circuit.measure(creg[0], creg[0])
3 circuit.measure(creg[1], creg[1])
```

ms

`m(theta, qubits)`

[GitHub ↗](#)

Apply `MSGate`

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- theta (*ParameterExpression* *float* $\times$) – The angle of the rotation.
- qubits (*Sequence* $\times$ *Qubit* *QuantumRegister* *int* $\times$ *slice* $\times$ *Sequence* $\times$ *Qubit* *int* $\times$ *int*) – The qubits to apply the gate to.

Returns

A handle to the instructions created.

Return type

InstructionSet

p

`p(theta, qubit)`

[GitHub ↗](#)

Apply `PhaseGate`

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- theta (*ParameterExpression* *float* $\times$) – The angle of the rotation.
- qubit (*Qubit* *QuantumRegister* *int* $\times$ *slice* $\times$ *Sequence* $\times$ *Qubit* *int* $\times$ *int*) – The qubit(s) to apply the gate to.

Returns

A handle to the instructions created.

Return type

InstructionSet

pauli

`pauli(pauli_string, qubits)`

[GitHub ↗](#)

Apply `PauliGate`

Parameters

- pauli_string (*str* $\times$) – A string representing the Pauli operator to apply, e.g. 'XX'.
- qubits (*Sequence* $\times$ *Qubit* *QuantumRegister* *int* $\times$ *slice* $\times$ *Sequence* $\times$ *Qubit* *int* $\times$ *int*) – The qubits to apply this gate to.

Returns

A handle to the instructions created.

Return type

InstructionSet

prepare_state

`prepare_state(state, qubits=None, label=None, normalize=False)`

[GitHub ↗](#)

Prepare qubits in a specific state.

This class implements a state preparing unitary. Unlike `Initialize()` it does not reset the qubits first.

Parameters

- state (*Statevector* | *Sequence**complex* $\times$ | *str* $\times$ *int* $\times$) –
The state to initialize to, can be either of the following.
 - Statevector or vector of complex amplitudes to initialize to.
 - Labels of basis states of the Pauli eigenstates Z, X, Y. See `Statevector.from_label()`. Notice the order of the labels is reversed with respect to the qubit index to be applied to. Example label '01' initializes the qubit zero to $|1\rangle$ and the qubit one to $|0\rangle$.
 - An integer that is used as a bitmap indicating which qubits to initialize to $|1\rangle$. Example: setting params to 5 would initialize qubit 0 and qubit 2 to $|1\rangle$ and qubit 1 to $|0\rangle$.
- qubits (*Sequence**QubitSpecifier* | *None*) – Qubits to initialize. If `None` the initialization is applied to all qubits in the circuit.
- label (*str* $\times$ | *None*) – An optional label for the gate
- normalize (*bool* $\times$) – Whether to normalize an input array to a unit vector.

Returns

A handle to the instruction that was just initialized

Return type

InstructionSet

Examples

Prepare a qubit in the state $(|0\rangle - |1\rangle) / \sqrt{2}$.

```
1 import numpy as np
2 from qiskit import QuantumCircuit
3
4 circuit = QuantumCircuit(1)
5 circuit.prepare_state((1/np.sqrt(2), -1/np.sqrt(2)), 0)
6 circuit.draw()
```

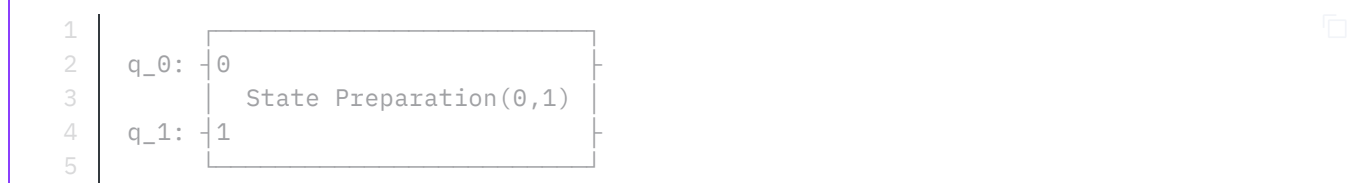
output:



Prepare from a string two qubits in the state $|10\rangle$. The order of the labels is reversed with respect to qubit index. More information about labels for basis states are in `Statevector.from_label()`

```
1 import numpy as np
2 from qiskit import QuantumCircuit
3
4 circuit = QuantumCircuit(2)
5 circuit.prepare_state('01', circuit.qubits)
6 circuit.draw()
```

output:



Initialize two qubits from an array of complex amplitudes .. code-block:

```
1 import numpy as np
2 from qiskit import QuantumCircuit
3
4 circuit = QuantumCircuit(2)
5 circuit.prepare_state([0, 1/np.sqrt(2), -1.j/np.sqrt(2), 0], circuit.qubits)
6 circuit.draw()
```

output:



r

`r(theta, phi, qubit)`

[GitHub ↗](#)

Apply `RGate`

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- theta (*ParameterExpression* *float* $\times$) – The angle of the rotation.
- phi (*ParameterExpression* *float* $\times$) – The angle of the axis of rotation in the x-y plane.
- qubit (*Qubit* *QuantumRegister* *int* $\times$ *slice* $\times$ *Sequence* $\times$ *Qubit* *int* $\times$ *int*) – The qubit(s) to apply the gate to.

Returns

A handle to the instructions created.

Return type

InstructionSet

rccx

`rccx(control_qubit1, control_qubit2, control_qubit3, target_qubit)`

[GitHub ↗](#)

Apply `RC3XGate`

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- control_qubit1 (*Qubit* *QuantumRegister* *int* $\times$ *slice* $\times$ *Sequence* $\times$ *Qubit* *int* $\times$ *int*) – The qubit(s) used as the first control.
- control_qubit2 (*Qubit* *QuantumRegister* *int* $\times$ *slice* $\times$ *Sequence* $\times$ *Qubit* *int* $\times$ *int*) – The qubit(s) used as the second control.
- control_qubit3 (*Qubit* *QuantumRegister* *int* $\times$ *slice* $\times$ *Sequence* $\times$ *Qubit* *int* $\times$ *int*) – The qubit(s) used as the third control.
- target_qubit (*Qubit* *QuantumRegister* *int* $\times$ *slice* $\times$ *Sequence* $\times$ *Qubit* *int* $\times$ *int*) – The qubit(s) targeted by the gate.

Returns

A handle to the instructions created.

Return type

InstructionSet

rcx

`rcx(control_qubit1, control_qubit2, target_qubit)`

[GitHub ↗](#)

Apply `RCCXGate`

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- `control_qubit1` (`Qubit` `QuantumRegister` `int` γ `slice` γ `Sequence` γ `Qubit` `int` γ) – The qubit(s) used as the first control.
- `control_qubit2` (`Qubit` `QuantumRegister` `int` γ `slice` γ `Sequence` γ `Qubit` `int` γ) – The qubit(s) used as the second control.
- `target_qubit` (`Qubit` `QuantumRegister` `int` γ `slice` γ `Sequence` γ `Qubit` `int` γ) – The qubit(s) targeted by the gate.

Returns

A handle to the instructions created.

Return type

`InstructionSet`

reset

`reset(qubit)`

[GitHub ↗](#)

Reset the quantum bit(s) to their default state.

Parameters

`qubit` (`Qubit` `QuantumRegister` `int` γ `slice` γ `Sequence` γ `Qubit` `int` γ) – qubit(s) to reset.

Returns

handle to the added instruction.

Return type

`qiskit.circuit.InstructionSet`

rv

`rv(vx, vy, vz, qubit)`

[GitHub ↗](#)

Apply `RVGate`

For the full matrix form of this gate, see the underlying gate documentation.

Rotation around an arbitrary rotation axis v , where $|v|$ is the angle of rotation in radians.

Parameters

- `vx` (`ParameterExpression` `float` γ) – x-component of the rotation axis.
- `vy` (`ParameterExpression` `float` γ) – y-component of the rotation axis.
- `vz` (`ParameterExpression` `float` γ) – z-component of the rotation axis.
- `qubit` (`Qubit` `QuantumRegister` `int` γ `slice` γ `Sequence` γ `Qubit` `int` γ) – The qubit(s) to apply the gate to.

Returns

A handle to the instructions created.

Return type

`InstructionSet`

rx

`rx(theta, qubit, label=None)`

[GitHub ↗](#)

Apply `RXGate`

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- `theta` (`ParameterValueType`) – The rotation angle of the gate.
- `qubit` (`QubitSpecifier`) – The qubit(s) to apply the gate to.
- `label` (`str` γ `None`) – The string label of the gate in the circuit.

Returns

A handle to the instructions created.

Return type

`InstructionSet`

rxx

`rxx(theta, qubit1, qubit2)`

[GitHub ↗](#)

Apply `RRXGate`

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- `theta` (`ParameterExpression` `float` γ) – The angle of the rotation.
- `qubit1` (`Qubit` `QuantumRegister` `int` γ `slice` γ `Sequence` γ `Qubit` `int` γ) – The qubit(s) to apply the gate to.
- `qubit2` (`Qubit` `QuantumRegister` `int` γ `slice` γ `Sequence` γ `Qubit` `int` γ) – The qubit(s) to apply the gate to.

Returns

A handle to the instructions created.

Return type

`InstructionSet`

ry

`ry(theta, qubit, label=None)`

[GitHub ↗](#)

Apply `RYGate`

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- `theta` (`ParameterValueType`) – The rotation angle of the gate.
- `qubit` (`QubitSpecifier`) – The qubit(s) to apply the gate to.
- `label` (`str` γ `None`) – The string label of the gate in the circuit.

Returns

A handle to the instructions created.

Return type

`InstructionSet`

ryy

`ryy(theta, qubit1, qubit2)`

[GitHub ↗](#)

Apply `RRYGate`

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- `theta` (`ParameterExpression` `float` γ) – The rotation angle of the gate.
- `qubit1` (`Qubit` `QuantumRegister` `int` γ `slice` γ `Sequence` γ `Qubit` `int` γ) – The qubit(s) to apply the gate to.
- `qubit2` (`Qubit` `QuantumRegister` `int` γ `slice` γ `Sequence` γ `Qubit` `int` γ) – The qubit(s) to apply the gate to.

Returns

A handle to the instructions created.

Return type

`InstructionSet`

rz

`rz(phi, qubit)`

[GitHub ↗](#)

Apply `RZGate`

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- `phi` (`ParameterExpression` `float` γ) – The rotation angle of the gate.
- `qubit` (`Qubit` `QuantumRegister` `int` γ `slice` γ `Sequence` γ `Qubit` `int` γ) – The qubit(s) to apply the gate to.

Returns

A handle to the instructions created.

Return type

`InstructionSet`

rxz

`rxz(theta, qubit1, qubit2)`

[GitHub ↗](#)

Apply `RRXGate`

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- `theta` (`ParameterExpression` `float` γ) – The rotation angle of the gate.
- `qubit1` (`Qubit` `QuantumRegister` `int` γ `slice` γ `Sequence` γ `Qubit` `int` γ) – The qubit(s) to apply the gate to.
- `qubit2` (`Qubit` `QuantumRegister` `int` γ `slice` γ `Sequence` γ `Qubit` `int` γ) – The qubit(s) to apply the gate to.

Returns

A handle to the instructions created.

Return type

`InstructionSet`

rzx

`rzx(theta, qubit1, qubit2)`

[GitHub ↗](#)

Apply `RZXGate`

For the full matrix form of this gate, see the underlying gate documentation.

Parameters

- `theta` (`ParameterExpression` `float` γ) – The rotation angle of the gate.

- InstructionSev

InstructionSev

InstructionSev

See also

SW2

InstructionSev

Return type

InstructionSev

InstructionSev

• **body** (*Optional*[QuantumCircuit](#)) – The loop body to be repeatedly executed. Omit this to use the context-manager mode.

• **qubits** (*Optional*[Sequence](#)[QubitSpecifier](#)) – The circuit qubits over which the loop body should be run. Omit this to use the context-manager mode.

• **clbits** (*Optional*[Sequence](#)[CbitSpecifier](#)) – The circuit clbits over which the loop body should be run. Omit this to use the context-manager mode.

• **label** (*Optional*[str](#) *>*) – The string label of the instruction in the circuit.

Returns

depending on the call signature, either a context manager for creating the for loop (it will automatically be added to the circuit at the end of the block), or an [InstructionSet](#) handle to the appended loop operation.

Return type

[InstructionSet](#) or [ForLoopContext](#)

Raises

CircuitError – if an incorrect calling convention is used.

if_else

```
[if_else(condition, true_body, false_body, qubits, clbits, label=None)]
```

[GitHub](#) *>*

Apply [IfElseOp](#)

Note

This method does not have an associated context-manager form, because it is already handled by the [if_test\(\)](#) method. You can use the [else](#) part of that with something such as:

```
1 | from qiskit.circuit import QuantumCircuit, Qubit, Cbit
2 | bits = [Qubit(), Qubit(), Cbit()]
3 | qc = QuantumCircuit(bits)
4 | qc.h(0)
5 | qc.cx(0, 1)
6 | qc.measure(0, 0)
7 | with qc.if_test((bits[2], 0)) as else_:
8 |     qc.h(0)
9 |     with else_:
10 |         qc.x(0)
```

Parameters

- **condition** (*tuple* *>* / *ClassicalRegister*, *int* *>* / *tuple* *>* / *Cbit*, *int* *>* / *tuple* *>* / *Cbit*, *bool* *>*) – A condition to be evaluated in real time at circuit execution, which, if true, will trigger the evaluation of [true_body](#). Can be specified as either a tuple of a [ClassicalRegister](#) to be tested for equality with a given [int](#) or as a tuple of a [Cbit](#) to be compared to either a [bool](#) or an [int](#).
- **true_body** (*QuantumCircuit*) – The circuit body to be run if [condition](#) is true.
- **false_body** (*QuantumCircuit*) – The circuit to be run if [condition](#) is false.
- **qubits** (*Sequence*[QubitSpecifier](#)) – The circuit qubits over which the if/else should be run.
- **clbits** (*Sequence*[CbitSpecifier](#)) – The circuit clbits over which the if/else should be run.
- **label** (*str* *>* / *None*) – The string label of the instruction in the circuit.

Raises

CircuitError – If the provided condition references Clbits outside the enclosing circuit.

Returns

A handle to the instruction created.

Return type

[InstructionSet](#)

if_test

```
[if_test(condition: tuple(ClassicalRegister | Cbit, int) -> IfContext)
```

```
[if_test(condition: tuple(ClassicalRegister | Cbit, int), true_body: QuantumCircuit, qubits: Sequence[Qubit | QuantumRegister | int | slice | Sequence[Qubit | int]], clbits: Sequence[Cbit | ClassicalRegister | int | slice | Sequence[Cbit | int]], *, label: str | None = None) -> InstructionSet]
```

[GitHub](#) *>*

Create an [if](#) statement on this circuit.

There are two forms for calling this function. If called with all its arguments (with the possible exception of [label](#)), it will create a [IfElseOp](#) with the given [true_body](#) and there will be no branch for the [false](#) condition (see also the [if_else\(\)](#) method). However, if [true_body](#) (and [qubits](#) and [clbits](#)) are not passed, then this acts as a context manager, which can be used to build [if](#) statements. The return value of the [if](#) statement is a chainable context manager, which can be used to create subsequent [if](#) blocks. In this form, you do not need to keep track of the qubits or clbits you are using, because the scope will handle it for you.

For example:

```
1 | from qiskit.circuit import QuantumCircuit, Qubit, Cbit
2 | bits = [Qubit(), Qubit(), Qubit(), Cbit(), Cbit()]
3 | qc = QuantumCircuit(bits)
4 |
5 | qc.h(0)
6 | qc.cx(0, 1)
7 | qc.measure(0, 0)
8 | qc.h(0)
9 | qc.cx(0, 1)
10 | qc.measure(0, 1)
11 |
12 | with qc.if_test((bits[3], 0)) as else_:
13 |     qc.x(2)
14 |     with else_:
15 |         qc.h(2)
16 |         qc.z(2)
```

Parameters

- **condition** (*Tuple*[Union](#)[ClassicalRegister](#), *Cbit*, *int* *>*) – A condition to be evaluated in real time during circuit execution, which, if true, will trigger the evaluation of [true_body](#). Can be specified as either a tuple of a [ClassicalRegister](#) to be tested for equality with a given [int](#), or as a tuple of a [Cbit](#) to be compared to either a [bool](#) or an [int](#).
- **true_body** (*Optional*[QuantumCircuit](#)) – The circuit body to be run if [condition](#) is true.
- **qubits** (*Optional*[Sequence](#)[QubitSpecifier](#)) – The circuit qubits over which the if/else should be run.
- **clbits** (*Optional*[Sequence](#)[CbitSpecifier](#)) – The circuit clbits over which the if/else should be run.
- **label** (*Optional*[str](#) *>*) – The string label of the instruction in the circuit.

Returns

depending on the call signature, either a context manager for creating the [if](#) block (it will automatically be added to the circuit at the end of the block), or an [InstructionSet](#) handle to the appended conditional operation.

Return type

[InstructionSet](#) or [IfContext](#)

Raises

- **CircuitError** – If the provided condition references Clbits outside the enclosing circuit.
- **CircuitError** – if an incorrect calling convention is used.

Returns

A handle to the instruction created.

switch

```
[switch(target: Cbit | ClassicalRegister | int | slice | Sequence[Cbit | int], cases: None, qubits: None, clbits: None, *, label: str | None) -> SwitchContext]
```

```
[switch(target: Cbit | ClassicalRegister | int | slice | Sequence[Cbit | int], cases: Iterable[Tuple[Any, QuantumCircuit]], qubits: Sequence[Qubit | QuantumRegister | int | slice | Sequence[Qubit | int]], clbits: Sequence[Cbit | ClassicalRegister | int | slice | Sequence[Cbit | int]], *, label: str | None) -> InstructionSet]
```

[GitHub](#) *>*

Create a [switch](#) [case](#) structure on this circuit.

There are two forms for calling this function. If called with all its arguments (with the possible exception of [label](#)), it will create a [SwitchCaseOp](#) with the given case structure. If [cases](#) (and [qubits](#) and [clbits](#)) are not passed, then this acts as a context manager, which will automatically build a [SwitchCaseOp](#) when the scope finishes. In this form, you do not need to keep track of the qubits or clbits you are using, because the scope will handle it for you.

Example usage:

```
1 | from qiskit.circuit import QuantumCircuit, ClassicalRegister, QuantumRegister
2 | qreg = QuantumRegister(3)
3 | creg = ClassicalRegister(3)
4 | qc = QuantumCircuit(qreg, creg)
5 | qc.h([0, 1, 2])
6 | qc.measure([0, 1, 2], [0, 1, 2])
7 |
8 | with qc.switch(creg) as case:
9 |     with case(0):
10 |         qc.x(0)
11 |         with case(1, 2):
12 |             qc.z(1)
13 |             with case(case.DEFAULT):
14 |                 qc.cx(0, 1)
```

Parameters

- **target** (*Union*[ClassicalRegister](#), *Cbit*) – The classical value to switch one. This must be integer-like.
- **cases** (*Iterable*[Tuple](#)[Any](#) *>*, *QuantumCircuit*) – A sequence of case specifiers. Each tuple defines one case body (the second item). The first item of the tuple can be either a single integer value, the special value [CASE_DEFAULT](#) or a tuple of several integer values. Each of the integer values will be tried in turn; control will then pass to the body corresponding to the first match. [CASE_DEFAULT](#) matches all possible values. Omit in context-manager form.
- **qubits** (*Sequence*[Qubit](#)) – The circuit qubits over which all case bodies execute. Omit in context-manager form.
- **clbits** (*Sequence*[Cbit](#)) – The circuit clbits over which all case bodies execute. Omit in context-manager form.
- **label** (*Optional*[str](#) *>*) – The string label of the instruction in the circuit.

Returns

If used in context-manager mode, then this should be used as a [with](#) resource, which will return an object that can be repeatedly entered to produce cases for the switch statement. If the full form is used, then this returns a handle to the instructions created.

Return type

[InstructionSet](#) or [SwitchCaseContext](#)

Raises

CircuitError – if an incorrect calling convention is used.

while_loop

```
[while_loop(condition: tuple(ClassicalRegister | Cbit, int) | expr.Expr, body: None, qubits: None, clbits: None, *, label: str | None) -> while_loopContext]
```

```
[while_loop(condition: tuple(ClassicalRegister | Cbit, int) | expr.Expr, body: QuantumCircuit, qubits: Sequence[Qubit | QuantumRegister | int | slice | Sequence[Qubit | int]], clbits: Sequence[Cbit | ClassicalRegister | int | slice | Sequence[Cbit | int]], *, label: str | None) -> InstructionSet]
```

[GitHub](#) *>*

Create a [while](#) loop on this circuit.

There are two forms for calling this function. If called with all its arguments (with the possible exception of [label](#)), it will create a [WhileLoopOp](#) with the given [body](#), if [body](#) (and [qubits](#) and [clbits](#)) are not passed, then this acts as a context manager, which will automatically build a [WhileLoopOp](#) when the scope finishes. In this form, you do not need to keep track of the qubits or clbits you are using, because the scope will handle it for you.

Example usage:


```
1 from qiskit.circuit import QuantumCircuit, Cbit, Qubit
2 bits = [Qubit(), Qubit(), Cbit()]
3 qc = QuantumCircuit(bits)
4
5 with qc.while_loop((bits[2], 0)):
6     qc.h(0)
7     qc.cx(0, 1)
8     qc.measure(0, 0)
```

Parameters

- condition** (*Tuple/Union*[ClassicalRegister](#), [Cbit](#), *int* *n*) – An equality condition to be checked prior to executing [body](#). The left-hand side of the condition must be a [ClassicalRegister](#) or a [Cbit](#) and the right-hand side must be an integer or boolean.
- body** (*Optional*[QuantumCircuit](#)) – The loop body to be repeatedly executed. Omit this to use the context-manager mode.
- qubits** (*Optional*[Sequence](#)[Qubit](#)) – The circuit qubits over which the loop body should be run. Omit this to use the context-manager mode.
- clbits** (*Optional*[Sequence](#)[Cbit](#)) – The circuit clbits over which the loop body should be run. Omit this to use the context-manager mode.
- label** (*Optional*[str](#) *n*) – The string label of the instruction in the circuit.

Returns

If used in context-manager mode, then this should be used as a [with](#) resource, which will infer the block content and operands on exit. If the full form is used, then this returns a handle to the instructions created.

Return type

InstructionSet or WhileLoopContext

Raises

CircuitError – if an incorrect calling convention is used.

Converting circuits to single objects

As discussed in [Methods to add general operations](#), you can convert a circuit to either an [Instruction](#) or a [Gate](#) using two helper methods.

to_instruction

```
to_instruction(parameter_map=None, label=None)
```

GitHub ↗

Create an [Instruction](#) out of this circuit.

See also

[Circuit.to_instruction\(\)](#)

The underlying driver of this method.

Parameters

- parameter_map** (*dict* *n* [Parameter](#), [ParameterValue](#)*Type*) / *None*) – For parameterized circuits, a mapping from parameters in the circuit to parameters to be used in the instruction. If *None*, existing circuit parameters will also parameterize the instruction.
- label** (*str* *n* / *None*) – Optional gate label.

Returns

a composite instruction encapsulating this circuit (can be decomposed back).

Return type

qiskit.circuit.Instruction

to_gate

```
to_gate(parameter_map=None, label=None)
```

GitHub ↗

Create a [Gate](#) out of this circuit. The circuit must act only qubits and contain only unitary operations.

See also

[Circuit.to_gate\(\)](#)

The underlying driver of this method.

Parameters

- parameter_map** (*dict* *n* [Parameter](#), [ParameterValue](#)*Type*) / *None*) – For parameterized circuits, a mapping from parameters in the circuit to parameters to be used in the gate. If *None*, existing circuit parameters will also parameterize the gate.
- label** (*str* *n* / *None*) – Optional gate label.

Returns

a composite gate encapsulating this circuit (can be decomposed back).

Return type

Gate

Helper mutation methods

There are two higher-level methods on [QuantumCircuit](#) for appending measurements to the end of a circuit. Note that by default, these also add an extra register.

measure_active

```
measure_active(inplace=True)
```

GitHub ↗

Adds measurement to all non-idle qubits. Creates a new [ClassicalRegister](#) with a size equal to the number of non-idle qubits being measured.

Returns a new circuit with measurements if *inplace=False*.

Parameters

inplace (*bool* *n*) – All measurements inplace or return new circuit.

Returns

Returns circuit with measurements when [inplace = False](#)

Return type

QuantumCircuit

measure_all

```
measure_all(inplace=True, add_bits=True)
```

GitHub ↗

Adds measurement to all qubits.

By default, adds new classical bits in a [ClassicalRegister](#) to store these measurements. If [add_bits=False](#), the results of the measurements will instead be stored in the already existing classical bits, with qubit [n](#) being measured into classical bit [n](#).

Returns a new circuit with measurements if [inplace=False](#)

Parameters

- inplace** (*bool* *n*) – All measurements inplace or return new circuit.
- add_bits** (*bool* *n*) – Whether to add new bits to store the results.

Returns

Returns circuit with measurements when [inplace=False](#)

Return type

QuantumCircuit

Raises

CircuitError – if [add_bits=False](#) but there are not enough classical bits.

There are two "subtractive" methods on [QuantumCircuit](#) as well. This is not a use-case that [QuantumCircuit](#) is designed for; typically you should just look to use [copy_empty_like\(\)](#) in place of [clear\(\)](#) and run [remove_final_measurements\(\)](#) as its transpiler-pass form [RemoveFinalMeasurements](#)

clear

```
clear()
```

GitHub ↗

Clear all instructions in self.

Clearing the circuits will keep the metadata and calibrations.

See also

[copy_empty_like\(\)](#)

A method to produce a new circuit with no instructions and all the same tracking of quantum and classical typed data, but without mutating the original circuit.

Return type

None

remove_final_measurements

```
remove_final_measurements(inplace=True)
```

GitHub ↗

Removes final measurements and barriers on all qubits if they are present. Deletes the classical registers that were used to store the values from these measurements that become idle as a result of this operation, and deletes classical bits that are referenced only by removed registers, or that aren't referenced at all but have become idle as a result of this operation.

Measurements and barriers are considered final if they are followed by no other operations (aside from other measurements or barriers).

Note

This method has rather complex behavior, particularly around the removal of newly idle classical bits and registers. It is much more efficient to avoid adding unnecessary classical data in the first place, rather than trying to remove it later.

See also

[RemoveFinalMeasurements](#)

A transpiler pass that removes final measurements and barriers. This does not remove the classical data. If this is your goal, you can call that with:

```
1 from qiskit.circuit import QuantumCircuit
2 from qiskit.transpiler.passes import RemoveFinalMeasurements
3
4 qc = QuantumCircuit(2, 2)
5 qc.h(0)
6 qc.cx(0, 1)
7 qc.barrier()
8 qc.measure([0, 1], [0, 1])
9
10 pass_ = RemoveFinalMeasurements()
11 just_bell = pass_(qc)
```

Parameters

inplace (*bool* *n*) – All measurements removed inplace or return new circuit.

Returns

Returns the resulting circuit when [inplace=False](#) else *None*.

Return type
QuantumCircuit

Manual calibration of instructions

`QuantumCircuit` can store `calibrations` of instructions that define the pulses used to run them on one particular hardware backend. You can

add_calibration

```
add_calibration(gate, qubits, schedule, params=None)
```

[GitHub ↗](#)

Register a low-level, custom pulse definition for the given gate.

⛔

Deprecated since version 1.3

The method `[qiskit.circuit.quantumcircuit.QuantumCircuit.add_calibration()]` is deprecated as of Qiskit 1.3. It will be removed in Qiskit 2.0. The entire Qiskit Pulse package is being deprecated and will be moved to the Qiskit Dynamics repository: <https://github.com/qiskit-community/qiskit-dynamics> ↗. Note that once removed, `[qiskit.circuit.quantumcircuit.QuantumCircuit.add_calibration()]` will have no alternative in Qiskit.

Parameters

- `gate` (`Union(Gate, str)`) – Gate information.
- `qubits` (`Union(int)` ↗, `Tuple(int)` ↗) – List of qubits to be measured.
- `schedule` (`Schedule`) – Schedule information.
- `params` (`Optional(List(Union(float)` ↗, `Parameter`))) – A list of parameters.

Raises

Exception ↗ – if the gate is of type string and params is None.

Return type

None

has_calibration_for

```
has_calibration_for(instruction)
```

[GitHub ↗](#)

Return True if the circuit has a calibration defined for the instruction context. In this case, the operation does not need to be translated to the device basis.

⛔

Deprecated since version 1.3

The method `[qiskit.circuit.quantumcircuit.QuantumCircuit.has_calibration_for()]` is deprecated as of Qiskit 1.3. It will be removed in Qiskit 2.0. The entire Qiskit Pulse package is being deprecated and will be moved to the Qiskit Dynamics repository: <https://github.com/qiskit-community/qiskit-dynamics> ↗. Note that once removed, `[qiskit.circuit.quantumcircuit.QuantumCircuit.has_calibration_for()]` will have no alternative in Qiskit.

Parameters

`instruction` (`CircuitInstruction` `tuple` ↗) –

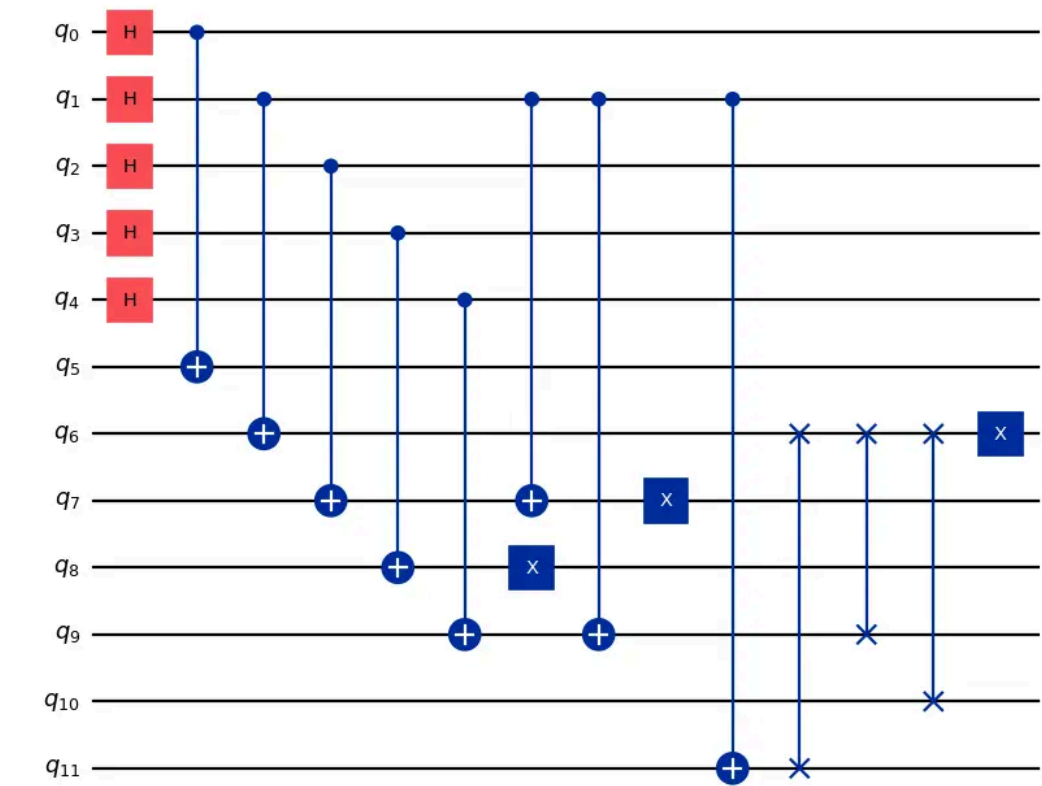
Circuit properties

Simple circuit metrics

When constructing quantum circuits, there are several properties that help quantify the “size” of the circuits, and their ability to be run on a noisy quantum device. Some of these, like number of qubits, are straightforward to understand, while others like depth and number of tensor components require a bit more explanation. Here we will explain all of these properties, and, in preparation for understanding how circuits change when run on actual devices, highlight the conditions under which they change.

Consider the following circuit:

```
1 from qiskit import QuantumCircuit
2 qc = QuantumCircuit(12)
3 for idx in range(5):
4     qc.h(idx)
5     qc.cx(idx, idx+5)
6
7 qc.cx(1, 7)
8 qc.x(8)
9 qc.cx(1, 9)
10 qc.x(7)
11 qc.cx(1, 11)
12 qc.swap(6, 11)
13 qc.swap(6, 9)
14 qc.swap(6, 10)
15 qc.x(6)
16 qc.draw('mpl')
```



From the plot, it is easy to see that this circuit has 12 qubits, and a collection of Hadamard, CNOT, X, and SWAP gates. But how to quantify this programmatically? Because we can do single-qubit gates on all the qubits simultaneously, the number of qubits in this circuit is equal to the `width()` of the circuit:

```
assert qc.width() == 12
```

We can also just get the number of qubits directly using `num_qubits`

```
assert qc.num_qubits == 12
```

⛔

Important

For a quantum circuit composed from just qubits, the circuit width is equal to the number of qubits. This is the definition used in quantum computing. However, for more complicated circuits with classical registers, and classically controlled gates, this equivalence breaks down. As such, from now on we will not refer to the number of qubits in a quantum circuit as the width.

It is also straightforward to get the number and type of the gates in a circuit using `count_ops()`

```
qc.count_ops()
```

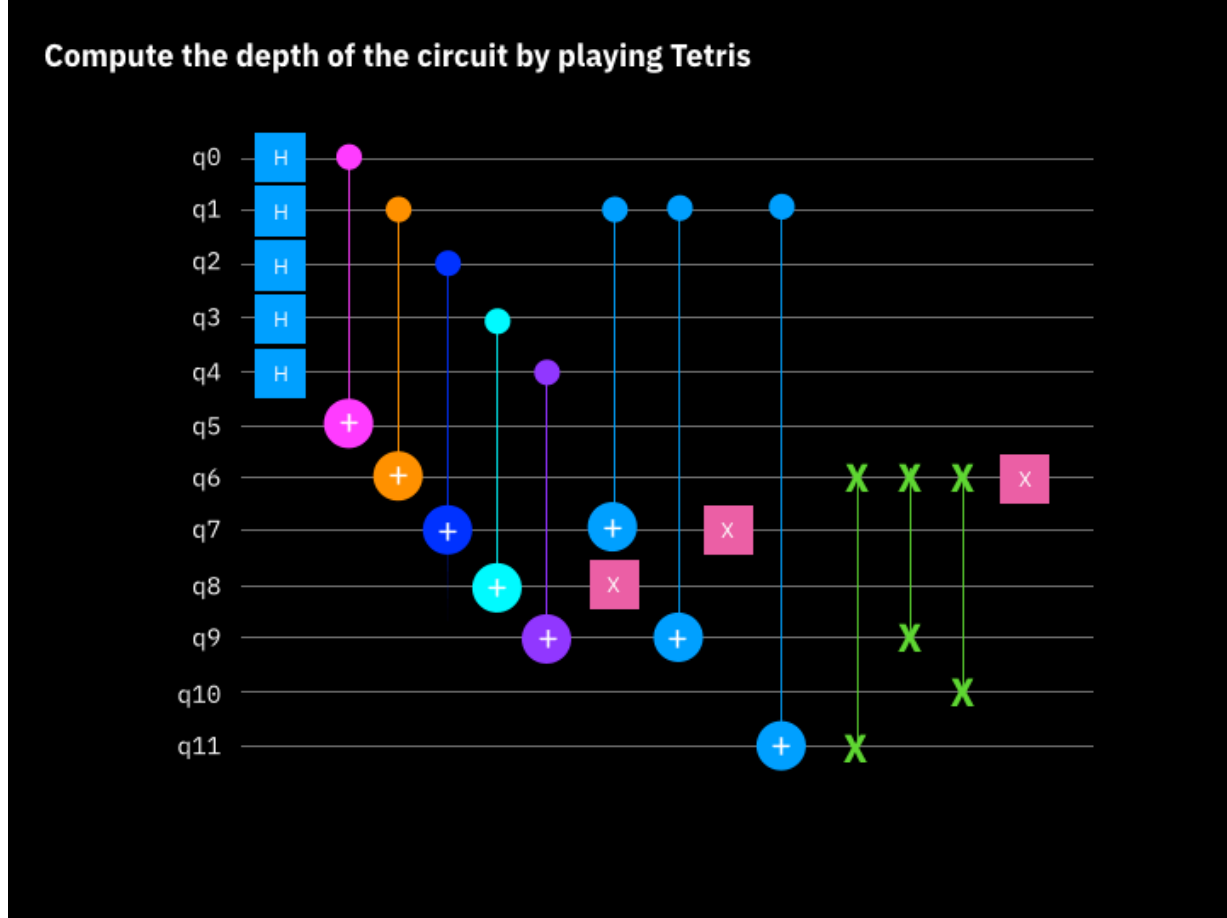
```
OrderedDict([('cx', 8), ('h', 5), ('x', 3), ('swap', 3)])
```

We can also get just the raw count of operations by computing the circuits `size()`

```
assert qc.size() == 19
```

A particularly important circuit property is known as the circuit `depth()`. The depth of a quantum circuit is a measure of how many “layers” of quantum gates, executed in parallel, it takes to complete the computation defined by the circuit. Because quantum gates take time to implement, the depth of a circuit roughly corresponds to the amount of time it takes the quantum computer to execute the circuit. Thus, the depth of a circuit is one important quantity used to measure if a quantum circuit can be run on a device.

The depth of a quantum circuit has a mathematical definition as the longest path in a directed acyclic graph (DAG). However, such a definition is a bit hard to grasp, even for experts. Fortunately, the depth of a circuit can be easily understood by anyone familiar with playing Tetris ↗. Let's see how to compute this graphically:



We can verify our graphical result using `QuantumCircuit.depth()`

```
assert qc.depth() == 9
```

count_ops

```
count_ops()
```

[GitHub ↗](#)

Count each operation kind in the circuit.

Returns

a breakdown of how many operations of each kind, sorted by amount.

Return type

OrderedDict

depth

```
depth(filter_function=<function QuantumCircuit.<lambda>>)
```

[GitHub ↗](#)

Return circuit depth (i.e., length of critical path).

⚠

Warning

This operation is not well defined if the circuit contains control-flow operations.

Parameters

`filter_function` (`Callable` ↗ `[CircuitInstruction]`, `bool` ↗) – A function to decide which instructions count to increase depth. Should take as a single positional input a `CircuitInstruction` Instructions for which the

function returns `False` are ignored in the computation of the circuit depth. By default, filters out "directives", such as `Barrier`.

Returns

Depth of circuit.

Return type

int ↗

Examples

Simple calculation of total circuit depth:

```
1 | from qiskit.circuit import QuantumCircuit
2 | qc = QuantumCircuit(4)
3 | qc.h(0)
4 | qc.cx(0, 1)
5 | qc.h(2)
6 | qc.cx(2, 3)
7 | assert qc.depth() == 2
```

Modifying the previous example to only calculate the depth of multi-qubit gates:

```
assert qc.depth(lambda instr: len(instr.qubits) > 1) == 1
```

get_instructions

`get_instructions(name)`

[GitHub ↗](#)

Get instructions matching name.

Parameters

`name (str ↗)` – The name of instruction to.

Returns

list of (instruction, qargs, cargs).

Return type

list ↗(tuple ↗)

num_connected_components

`num_connected_components(unitary_only=False)`

[GitHub ↗](#)

How many non-entangled subcircuits can the circuit be factored to.

Parameters

`unitary_only (bool ↗)` – Compute only unitary part of graph.

Returns

Number of connected components in circuit.

Return type

int ↗

num_nonlocal_gates

`num_nonlocal_gates()`

[GitHub ↗](#)

Return number of non-local gates (i.e. involving 2+ qubits). Conditional nonlocal gates are also included.

Return type

int ↗

num_tensor_factors

`num_tensor_factors()`

[GitHub ↗](#)

Computes the number of tensor factors in the unitary (quantum) part of the circuit only.

Notes

This is here for backwards compatibility, and will be removed in a future release of Qiskit. You should call `num_unitary_factors` instead.

Return type

int ↗

num_unitary_factors

`num_unitary_factors()`

[GitHub ↗](#)

Computes the number of tensor factors in the unitary (quantum) part of the circuit only.

Return type

int ↗

size

`size(filter_function=<function QuantumCircuit.<lambda>>)`

[GitHub ↗](#)

Returns total number of instructions in circuit.

Parameters

`filter_function (callable)` – a function to filter out some instructions. Should take as input a tuple of (Instruction, list(Qubit), list(Cbit)). By default, filters out "directives", such as barrier or snapshot.

Returns

Total number of gate operations.

Return type

int ↗

width

`width()`

[GitHub ↗](#)

Return number of qubits plus cbits in circuit.

Returns

Width of circuit.

Return type

int ↗

Accessing scheduling information

If a `QuantumCircuit` has been scheduled as part of a transpilation pipeline, the timing information for individual qubits can be accessed. The whole-circuit timing information is available through the `duration` unit and `op_start_times` attributes.

qubit_duration

`qubit_duration(*qubits)`

[GitHub ↗](#)

Return the duration between the start and stop time of the first and last instructions, excluding delays, over the supplied qubits. Its time unit is `self.unit`.

Parameters

`*qubits (Qubit | int ↗)` – Qubits within `self` to include.

Returns

Return the duration between the first start and last stop time of non-delay instructions

Return type

float ↗

qubit_start_time

`qubit_start_time(*qubits)`

[GitHub ↗](#)

Return the start time of the first instruction, excluding delays, over the supplied qubits. Its time unit is `self.unit`. Return 0 if there are no instructions over qubits

Parameters

- `*qubits` – Qubits within `self` to include. Integers are allowed for qubits, indicating
- `self.qubits`, (*indices of* –

Returns

Return the start time of the first instruction, excluding delays, over the qubits

Raises

`CircuitError` – if `self` is a not-yet scheduled circuit.

Return type

float ↗

qubit_stop_time

`qubit_stop_time(*qubits)`

[GitHub ↗](#)

Return the stop time of the last instruction, excluding delays, over the supplied qubits. Its time unit is `self.unit`. Return 0 if there are no instructions over qubits

Parameters

- `*qubits` – Qubits within `self` to include. Integers are allowed for qubits, indicating
- `self.qubits`, (*indices of* –

Returns

Return the stop time of the last instruction, excluding delays, over the qubits

Raises

`CircuitError` – if `self` is a not-yet scheduled circuit.

Return type

float ↗

`QuantumCircuit` also contains a small number of methods that are very `[Instruction]` like in detail. You may wish find better integration and more API support if you first convert your circuit to an `[Instruction]` `[to_instruction()]` or `[Gate]` `[to_gate()]` as appropriate, then call the corresponding method.

control

`[control(num_ctrl_qubits=1, label=None, ctrl_state=None, annotated=False)]`

[GitHub ↗](#)

Control this circuit on `[num_ctrl_qubits]` qubits.

Parameters

- `num_ctrl_qubits` (*int* γ) – The number of control qubits.
- `label` (*str* γ) – An optional label to give the controlled operation for visualization.
- `ctrl_state` (*str* γ or *int* γ) – The control state in decimal or as a bitstring (e.g. '111'). If None, use `[2**num_ctrl_qubits - 1]`
- `annotated` (*bool* γ) – Indicates whether the controlled gate should be implemented as an annotated gate.

Returns

The controlled version of this circuit.

Return type

`QuantumCircuit`

Raises

CircuitError – If the circuit contains a non-unitary operation and cannot be controlled.

inverse

`[inverse(annotated=False)]`

[GitHub ↗](#)

Invert (take adjoint of) this circuit.

This is done by recursively inverting all gates.

Parameters

`annotated` (*bool* γ) – indicates whether the inverse gate can be implemented as an annotated gate.

Returns

the inverted circuit

Return type

`QuantumCircuit`

Raises

CircuitError – if the circuit cannot be inverted.

Examples

input:



output:



power

`[power(power, matrix_power=False, annotated=False)]`

[GitHub ↗](#)

Raise this circuit to the power of `[power]`

If `[power]` is a positive integer and both `[matrix_power]` and `[annotated]` are `[False]` this implementation defaults to calling `[repeat]`. Otherwise, the circuit is converted into a gate, and a new circuit, containing this gate raised to the given power, is returned. The gate raised to the given power is implemented either as a unitary gate if `[annotated]` is `[False]` or as an annotated operation if `[annotated]` is `[True]`

Parameters

- `power` (*float* γ) – The power to raise this circuit to.
- `matrix_power` (*bool* γ) – indicates whether the inner power gate can be implemented as a unitary gate.
- `annotated` (*bool* γ) – indicates whether the inner power gate can be implemented as an annotated operation.

Raises

CircuitError – If the circuit needs to be converted to a unitary gate, but is not unitary.

Returns

A circuit implementing this circuit raised to the power of `[power]`

Return type

`QuantumCircuit`

repeat

`[repeat(reps, *, insert_barriers=False)]`

[GitHub ↗](#)

Repeat this circuit `[reps]` times.

Parameters

- `reps` (*int* γ) – How often this circuit should be repeated.
- `insert_barriers` (*bool* γ) – Whether to include barriers between circuit repetitions.

Returns

A circuit containing `[reps]` repetitions of this circuit.

Return type

`QuantumCircuit`

reverse_ops

`[reverse_ops()]`

[GitHub ↗](#)

Reverse the circuit by reversing the order of instructions.

This is done by recursively reversing all instructions. It does not invert (adjoint) any gate.

Returns

the reversed circuit.

Return type

`QuantumCircuit`

Examples

input:



output:



Visualization

Qiskit includes some drawing tools to give you a quick feel for what your circuit looks like. This tooling is primarily targeted at producing either a `Matplotlib` γ - or text-based drawing. There is also a lesser-featured LaTeX backend for drawing, but this is only for simple circuits, and is not as actively maintained.

[See also](#)

`[qiskit.visualization]`

The primary documentation for all of Qiskit's visualization tooling.

draw

`[draw(output=None, scale=None, filename=None, style=None, interactive=False, plot_barriers=True, reverse_bits=None, justify=None, vertical_compression='medium', idle_writes=None, with_layout=True, fold=None, ax=None, initial_state=False, cregbundle=None, wire_order=None, expr_len=30)]`

[GitHub ↗](#)

Draw the quantum circuit. Use the output parameter to choose the drawing format:

text: ASCII art TextDrawing that can be printed in the console.

mpl: images with color rendered purely in Python using matplotlib.

latex: high-quality images compiled via latex.

latex_source: raw uncompiled latex output.

Warning

Support for `[Expr]` nodes in conditions and `[SwitchCaseOp.target]` fields is preliminary and incomplete. The `[Text]` and `[mpl]` drawers will make a best-effort attempt to show data dependencies, but the LaTeX-based drawers will skip these completely.

Parameters

- `output` (*str* γ / *None*) – Select the output method to use for drawing the circuit. Valid choices are `[text]` `[mpl]` `[latex]` `[latex_source]`. By default, the `[text]` drawer is used unless the user config file (usually `[~/.qiskit/settings.conf]`) has an alternative backend set as the default. For example, `[circuit_drawer = latex]`. If the output kwarg is set, that backend will always be used over the default in the user config file.
- `scale` (*float* γ / *None*) – Scale of image to draw (shrink if `[< 1.0]`). Only used by the `[mpl]` `[latex]` and `[latex_source]` outputs. Defaults to `[1.0]`
- `filename` (*str* γ / *None*) – File path to save image to. Defaults to `[None]` (result not saved in a file).
- `style` (*dict* γ *str* γ / *None*) –
 - Style name, file name of style JSON file, or a dictionary specifying the style.
 - The supported style names are `[“top”]` (default), `[“top-dark”]` `[“clifford”]` `[“textbook”]` and `[“bw”]`
 - If given a JSON file, e.g. `[my_style.json]` or `[my_style]` (the `[.json]` extension may be omitted), this function attempts to load the style dictionary from that location. Note, that the JSON file must completely specify the visualization specifications. The file is searched for in

- `qiskit/visualization/circuit/styles` the current working directory, and the location specified in `../qiskit/settings.conf`
- If a dictionary, every entry overrides the default configuration. If the `name` key is given, the default configuration is given by that style. For example, `{name: "textbook", subfontsize = 5}` loads the "textbook" style and sets the subfontsize (e.g. the gate angles) to 5
 - If `None` the default style `"lap"` is used or, if given, the default style specified in `../qiskit/settings.conf`
- interactive** (*bool* γ) – When set to `True` show the circuit in a new window (for `mpl` this depends on the matplotlib backend being used supporting this). Note when used with either the text or the `latex_source` output type this has no effect and will be silently ignored. Defaults to `False`
 - reverse_bits** (*bool* γ / *None*) – When set to `True` reverse the bit order inside registers for the output visualization. Defaults to `False` unless the user config file (usually `../qiskit/settings.conf`) has an alternative value set. For example, `circuit.reverse_bits = True`
 - plot_barriers** (*bool* γ) – Enable/disable drawing barriers in the output circuit. Defaults to `True`
 - justify** (*str* γ / *None*) – Options are `"left"`, `"right"` or `"none"` (str). If anything else is supplied, left justified will be used instead. It refers to where gates should be placed in the output circuit if there is an option, `none` results in each gate being placed in its own column. Defaults to `left`.
 - vertical_compression** (*str* γ / *None*) – `"lap"`, `medium` or `low`. It merges the lines generated by the text output so the drawing will take less vertical room. Default is `medium`. Only used by the `text` output, will be silently ignored otherwise.
 - idle_wires** (*bool* γ / *None*) – Include idle wires (wires with no circuit elements) in output visualization. Default is `True` unless the user config file (usually `../qiskit/settings.conf`) has an alternative value set. For example, `circuit.idle_wires = False`
 - with_layout** (*bool* γ) – Include layout information, with labels on the physical layout. Default is `True`
 - fold** (*int* γ / *None*) – Sets pagination. It can be disabled using -1. In `text` sets the length of the lines. This is useful when the drawing does not fit in the console. If None (default), it will try to guess the console width using `shutil.get_terminal_size()`. However, if running in jupyter, the default line length is set to 80 characters. In `mpl` it is the number of (visual) layers before folding. Default is 25.
 - ax** (*Any* / *None*) – Only used by the mpl backend. An optional `matplotlib.axes.Axes` object to be used for the visualization output. If none is specified, a new matplotlib Figure will be created and used. Additionally, if specified there will be no returned Figure since it is redundant.
 - initial_state** (*bool* γ) – Adds $|0\rangle$ in the beginning of the qubit wires and $|0\rangle$ to classical wires. Default is `False`
 - cregbundle** (*bool* γ / *None*) – If set to `True` bundle classical registers. Default is `True` except for when `output` is set to `"text"`
 - wire_order** (*list* γ (*int* γ) / *None*) – A list of integers used to reorder the display of the bits. The list must have an entry for every bit with the bits in the range 0 to `num_qubits - num_classics`
 - expr_len** (*int* γ) – The number of characters to display if an `Expr` is used for the condition in a `ControlFlowOp`. If this number is exceeded, the string will be truncated at that number and `...` added to the end.

Returns

`TextDrawing` or `matplotlib.figure` or `PIL_image` or `str` γ :

- `TextDrawing` (if `output="text"`)

A drawing that can be printed as ascii art.

- `matplotlib.figure.Figure` (if `output="mpl"`)

A matplotlib figure object for the circuit diagram.

- `PIL_image` (if `output="latex"`)

An in-memory representation of the image of the circuit diagram.

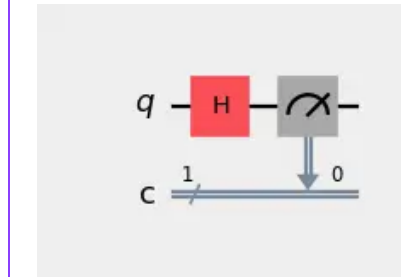
- `str` (if `output="latex_source"`)

The LaTeX source code for visualizing the circuit diagram.

- Raises
- VisualizationError** – when an invalid output method is selected
 - ImportError** γ – when the output methods requires non-installed libraries.

Example

```
1 from qiskit import QuantumRegister, ClassicalRegister, QuantumCircuit
2 qc = QuantumCircuit(1, 1)
3 qc.h(0)
4 qc.measure(0, 0)
5 qc.draw(output='mpl', style={'backgroundcolor': 'EEEEEE'})
```



In addition to the core `draw()` driver, there are two visualization-related helper methods, which are mostly useful for quickly unwrapping some inner instructions or reversing the qubit-labeling conventions in the drawing. For more general mutation, including basis-gate rewriting, you should use the transpiler `qiskit.transpiler`.

decompose

```
decompose(gates_to_decompose=None, reps=1)
```

GitHub γ

Call a decomposition pass on this circuit, to decompose one level (shallow decompose).

Parameters

- gates_to_decompose** (*str* γ (*TypeInstruction*) / *Sequence* (*str* γ (*TypeInstruction*) / *None*) – Optional subset of gates to decompose. Can be a gate type, such as `HGate` or a gate name, such as "h", or a gate label, such as "My H Gate", or a list of any combination of these. If a gate name is entered, it will decompose all gates with that name, whether the gates have labels or not. Defaults to all gates in the circuit.
- reps** (*int* γ) – Optional number of times the circuit should be decomposed. For instance `reps=2` equals calling `circuit.decompose().decompose()`

Returns

a circuit one level decomposed

Return type

QuantumCircuit

reverse_bits

```
reverse_bits()
```

GitHub γ

Return a circuit with the opposite order of wires.

The circuit is "vertically" flipped. If a circuit is defined over multiple registers, the resulting circuit will have the same registers but with their order flipped.

This method is useful for converting a circuit written in little-endian convention to the big-endian equivalent, and vice versa.

Returns

the circuit with reversed bit order.

Return type

QuantumCircuit

Examples

Input:

```
1
2 a_0: ──┐── H ──┐──
3       │      │
4 a_1: ──┴── X ──┐──
5       │      │
6 a_2: ──┴── X ──┐──
7       │      │
8 b_0: ──┴── X ──┐──
9       │      │
10      │      │
11      └── X ──┘
```

Output:

```
1
2 b_0: ──┐── X ──┐──
3       │      │
4 b_1: ──┴── X ──┐──
5       │      │
6 a_0: ──┴── X ──┐──
7       │      │
8 a_1: ──┴── X ──┐──
9       │      │
10      │      │
11      └── H ──┘
```

Internal utilities

These functions are not intended for public use, but were accidentally left documented in the public API during the 1.0 release. They will be removed in Qiskit 2.0, but will be supported until then.

cast

```
static cast(value, type)
```

GitHub γ

Best effort to cast value to type. Otherwise, returns the value.

🔴 **Deprecated since version 1.2**

The method `qiskit.circuit.quantumcircuit.QuantumCircuit.cast()` is deprecated as of qiskit 1.2. It will be removed in the 2.0 release. This method is only used as an internal helper and will be removed with no replacement.

Parameters

- value** (*S*) –
- type_** (*Callable* γ (*l*..., *T*)) –

Return type

S | *T*

cbit_argument_conversion

```
cbit_argument_conversion(cbit_representation)
```

GitHub γ

Converts several classical bit representations (such as indexes, range, etc.) into a list of classical bits.

🔴 **Deprecated since version 1.2**

The method `qiskit.circuit.quantumcircuit.QuantumCircuit.cbit_argument_conversion()` is deprecated as of qiskit 1.2. It will be removed in the 2.0 release. This method is only used as an internal helper and will be removed with no replacement.

Parameters

`cbit_representation` (*Cbit* (*ClassicalRegister* (*int* γ (*slice* γ (*Sequence* γ (*Cbit* (*int* γ))))) – Representation to expand.

Returns

A list of tuples where each tuple is a classical bit.

Return type

list >[qiskit.circuit.classicalregister.Cbit]

cls_instances

classmethod `cls_instances()`

GitHub >

Return the current number of instances of this class, useful for auto naming.

⛔ Deprecated since version 1.2

The method `qiskit.circuit.quantumcircuit.QuantumCircuit.cls_instances()` is deprecated as of qiskit 1.2. It will be removed in the 2.0 release. This method is only used as an internal helper and will be removed with no replacement.

Return type

int >

cls_prefix

classmethod `cls_prefix()`

GitHub >

Return the prefix to use for auto naming.

⛔ Deprecated since version 1.2

The method `qiskit.circuit.quantumcircuit.QuantumCircuit.cls_prefix()` is deprecated as of qiskit 1.2. It will be removed in the 2.0 release. This method is only used as an internal helper and will be removed with no replacement.

Return type

str >

qbit_argument_conversion

`qbit_argument_conversion(qubit_representation)`

GitHub >

Converts several qubit representations (such as indexes, range, etc.) into a list of qubits.

⛔ Deprecated since version 1.2

The method `qiskit.circuit.quantumcircuit.QuantumCircuit.qbit_argument_conversion()` is deprecated as of qiskit 1.2. It will be removed in the 2.0 release. This method is only used as an internal helper and will be removed with no replacement.

Parameters

qubit_representation (*Qubit* | *QuantumRegister* | *int* > | *slice* > | *Sequence* > | *Qubit* (*int* >)) – Representation to expand.

Returns

The resolved instances of the qubits.

Return type

list >[qiskit.circuit.quantumregister.Qubit]

Parameters

- regs (*Register* (*int* > | *Sequence*(*Bit*)) –
- name (*str* > | *None*) –
- global_phase (*ParameterValueType*) –
- metadata (*dict* > | *None*) –
- inputs (*Iterable*(*Expr*.*Var*)) –
- captures (*Iterable*(*Expr*.*Var*) –
- declarations (*Mapping*[(*Expr*.*Var*, *Expr*.*Expr*) | *Iterable*[(*Expr*.*Var*, *Expr*.*Expr*)]) –